

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 4:

Diseño Relacional, Optimización y Evolución del Esquema

Arquitectura de Datos

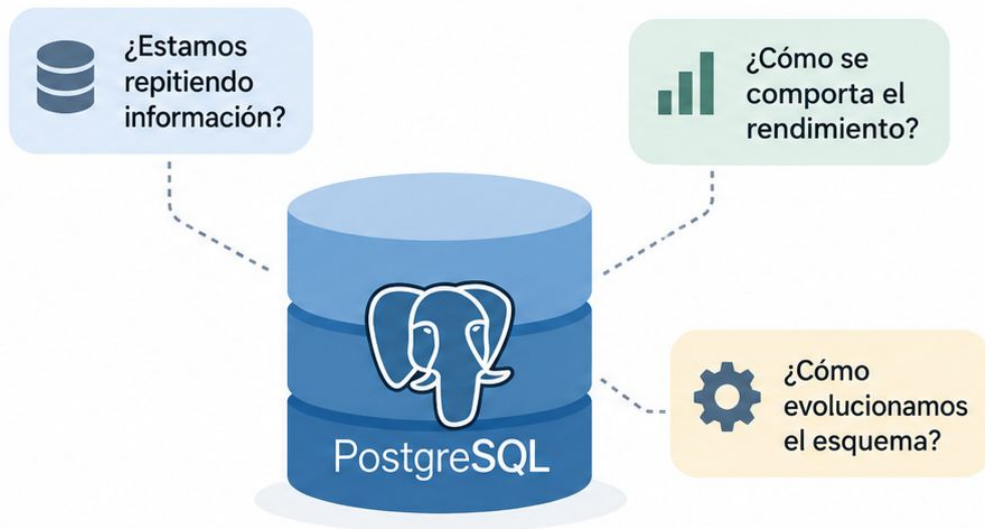
Ya tenemos una base que funciona

En la clase anterior logramos:

- crear estructuras;
- cargar y modificar datos;
- realizar consultas;
- combinar tablas;
- calcular métricas.

Ahora aparecen nuevas preguntas:

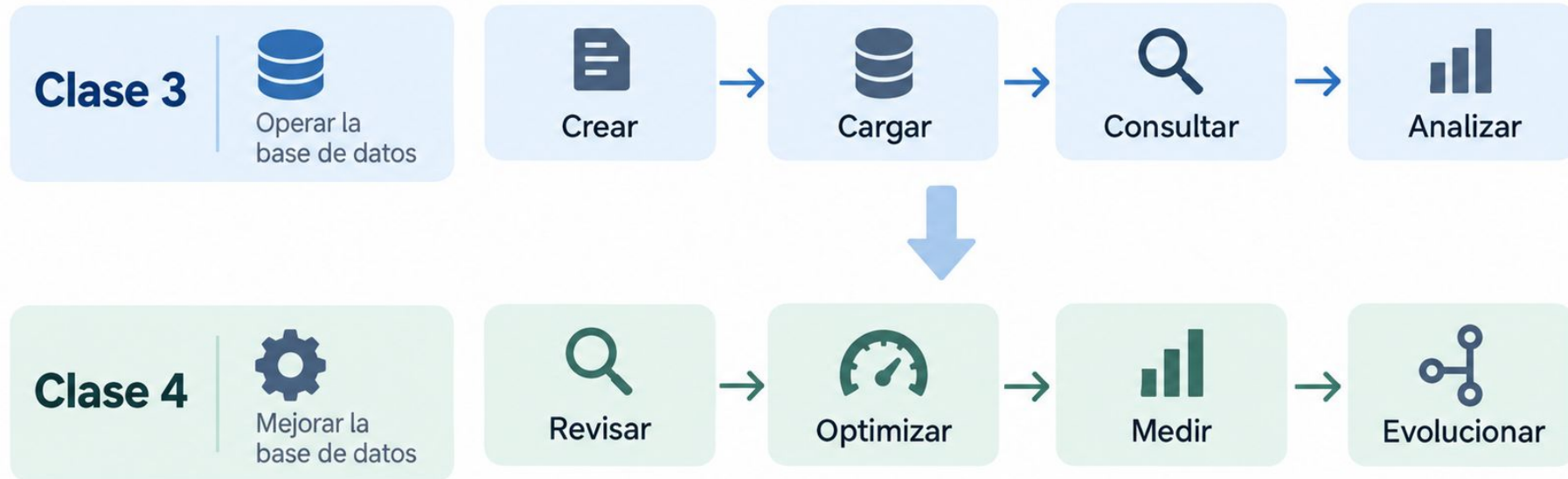
- ¿la estructura está bien diseñada?;
- ¿estamos repitiendo información?;
- ¿qué ocurre cuando crecen los datos?;
- ¿cómo incorporamos nuevos requerimientos?



Que una base funcione no significa necesariamente que esté bien diseñada ni optimizada.

Hasta ahora utilizamos la base

Hoy empezamos a evaluarla



Pasamos de aprender a utilizar el modelo a preguntarnos cómo mejorarlo.

Objetivos de la clase

Mejorar el diseño, el acceso y la evolución de una base relacional

Vamos a aprender a:

- reconocer dependencias y redundancia;
- normalizar estructuras;
- analizar desnormalización;
- utilizar índices;
- comprender selectividad y costos;
- analizar planes de ejecución;
- utilizar EXPLAIN y EXPLAIN ANALYZE;
- utilizar vistas;
- representar información variable mediante JSONB;
- evolucionar el esquema mediante migraciones.



Diseñar una base también implica pensar cómo será utilizada y cómo cambiará con el tiempo.

Seguimos con el mismo sistema

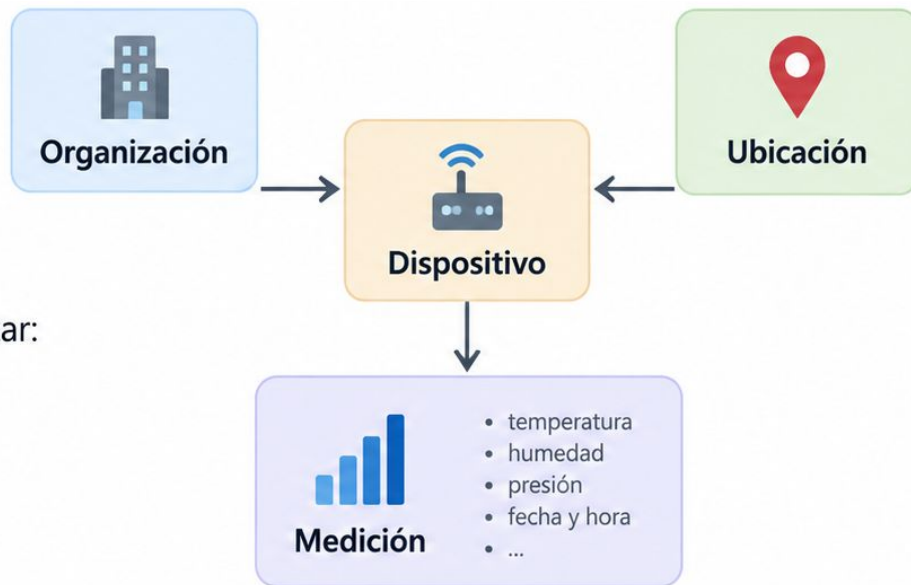
Monitoreo de ambientes

Trabajaremos con la base ya implementada:

- organizaciones;
- ubicaciones;
- dispositivos;
- mediciones.

Durante la clase utilizaremos esta estructura para analizar:

- problemas de diseño;
- consultas frecuentes;
- rendimiento;
- información variable;
- nuevos requerimientos.



No vamos a diseñar nuevamente el sistema. Vamos a revisar cómo puede mejorar.

Arquitectura de Datos

Agenda del día

- Dependencias y redundancia.
- Normalización: 1FN, 2FN y 3FN.
- Desnormalización justificada.
- Índices simples y compuestos.
- Selectividad y costo de mantenimiento.
- Planes de ejecución.
- EXPLAIN y EXPLAIN ANALYZE.
- Vistas y vistas materializadas.
- JSON y JSONB.
- Columnas, relaciones o JSONB.
- Migraciones y evolución del esquema.
- Práctica guiada.



Vamos a revisar la base desde tres perspectivas: estructura, rendimiento y evolución.

1

Dependencias y Redundancia

ID	Cliente	Ciudad	Producto	Precio
101	Ana López	Madrid	Portátil	1.200 €
102	Ana López	Madrid	Ratón	25 €
103	Ana López	Madrid	Monitor	300 €
104	Carlos Díaz	Barcelona	Teclado	45 €
105	Carlos Díaz	Barcelona	Monitor	300 €
106	Lucía Torres	Valencia	Portátil	1.200 €
107	Lucía Torres	Valencia	Ratón	25 €
108	Lucía Torres	Valencia	Webcam	90 €

Funcionar no alcanza

También necesitamos revisar cómo están organizados los datos

Una estructura puede aceptar datos y responder consultas, pero aun así repetir información innecesariamente.

Supongamos una tabla de mediciones que también almacena:

- nombre del dispositivo;
- nombre de la ubicación;
- nombre de la organización.

Cada medición vuelve a repetir esos mismos datos.

medicion_id	timestamp	valor	unidad	dispositivo	ubicación	organización
1	2026-09-01 10:00	24.1	°C	Sensor Aula 1	Aula 1	Escuela Norte
2	2026-09-01 10:05	24.3	°C	Sensor Aula 1	Aula 1	Escuela Norte
3	2026-09-01 10:10	24.0	°C	Sensor Aula 1	Aula 1	Escuela Norte
4	2026-09-01 10:15	23.8	°C	Sensor Aula 1	Aula 1	Escuela Norte
5	2026-09-01 10:20	24.2	°C	Sensor Aula 1	Aula 1	Escuela Norte
6	2026-09-01 10:25	24.1	°C	Sensor Aula 1	Aula 1	Escuela Norte
7	2026-09-01 10:30	24.4	°C	Sensor Aula 1	Aula 1	Escuela Norte
8	2026-09-01 10:35	24.0	°C	Sensor Aula 1	Aula 1	Escuela Norte
...	...	...	...	...	...	...



La misma información puede terminar almacenada miles o millones de veces.

¿Qué ocurre cuando algo cambia?

La redundancia también debe mantenerse

Supongamos:

ubicacion_id = 4

ubicacion_nombre = "Aula 204"

Luego cambia a:

"Laboratorio IoT"

Si el nombre está repetido en muchas filas debemos actualizar todas.

Si alguna queda sin modificar, aparecen valores diferentes para una misma ubicación.

medicion_id	timestamp	valor	unidad	ubicacion_id	ubicacion_nombre
101	2026-09-01 09:00	23.5	°C	4	Laboratorio IoT
102	2026-09-01 09:05	23.8	°C	4	Laboratorio IoT
103	2026-09-01 09:10	24.1	°C	4	Laboratorio IoT
104	2026-09-01 09:15	23.9	°C	4	Aula 204
105	2026-09-01 09:20	24.0	°C	4	Laboratorio IoT
106	2026-09-01 09:25	23.7	°C	4	Laboratorio IoT
107	2026-09-01 09:30	23.6	°C	4	Laboratorio IoT

Valor actualizado

Valor sin actualizar



Duplicar información también duplica la responsabilidad de mantenerla consistente.

Otros problemas de redundancia

La estructura condiciona las operaciones

Insertión

Si una ubicación solamente existe dentro de mediciones, ¿cómo registramos una ubicación que todavía no posee mediciones?

Tabla de mediciones

medicion_id	timestamp	valor	unidad	ubicacion_id	ubicacion_nombre
101	2026-09-01 09:00	23.5	°C	4	Laboratorio IoT
102	2026-09-01 09:05	23.8	°C	4	Laboratorio IoT
103	2026-09-01 09:10	24.1	°C	4	Laboratorio IoT
-	-	-	-	-	-

No se puede registrar "Aula 205" sin mediciones

Eliminación

Si eliminamos la última medición de un dispositivo, ¿podríamos perder también información sobre ese dispositivo?

Tabla de mediciones

medicion_id	timestamp	valor	unidad	dispositivo_id
201	2026-09-01 10:00	25.1	°C	10
202	2026-09-01 10:05	25.3	°C	10
203	2026-09-01 10:10	25.0	°C	10

Se elimina la última medición

Información del dispositivo

dispositivo_id: 10
nombre: Sensor A
tipo: Temperatura
ubicacion: Laboratorio IoT

dispositivo_id: 10
nombre: Sensor A
tipo: Temperatura
ubicacion: Laboratorio IoT

También se pierde la información del dispositivo



Una mala organización puede hacer que insertar o eliminar un dato afecte información que representa otra cosa.

Dependencias

Algunos valores determinan otros

Podemos expresar:

$A \rightarrow B$

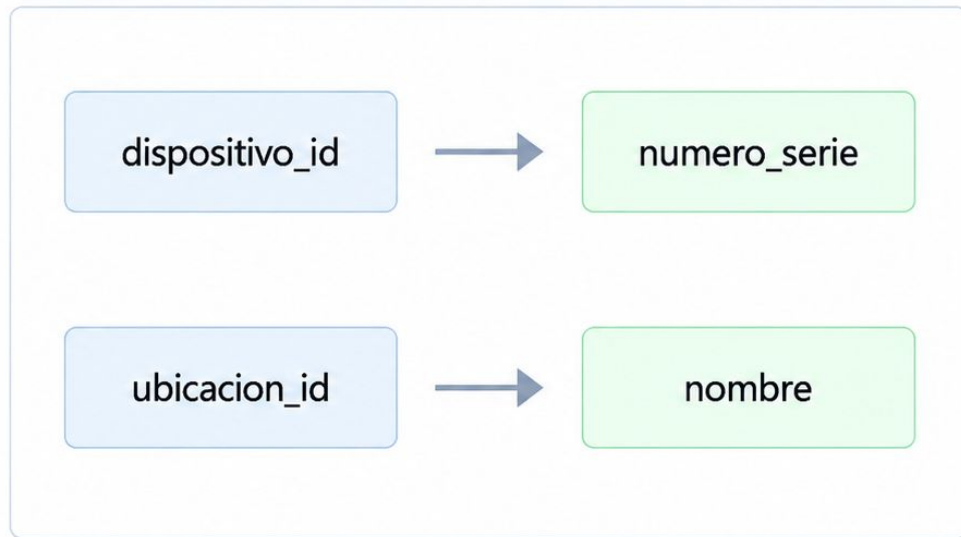
A determina B.

Ejemplos:

$\text{dispositivo_id} \rightarrow \text{numero_serie}$

$\text{ubicacion_id} \rightarrow \text{nombre}$

Conociendo el identificador podemos determinar el valor asociado.



Las dependencias ayudan a reconocer qué información describe realmente a cada elemento.

Una cadena de dependencias

Un atributo puede depender de otro de manera indirecta

dispositivo_id → ubicacion_id
ubicacion_id → ubicacion_nombre

Entonces:

dispositivo_id

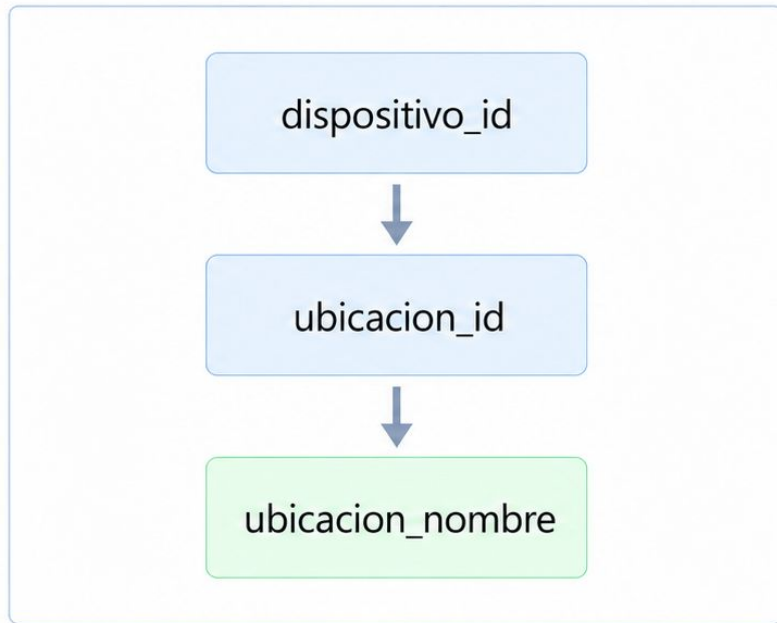


ubicacion_id



ubicacion_nombre

El nombre de la ubicación describe a la ubicación, no directamente al dispositivo.



Reconocer las dependencias ayuda a decidir dónde debería almacenarse cada dato.

Tenemos un problema de diseño

Necesitamos reorganizar la estructura

Estructura inicial:

Medición

+ datos del dispositivo

+ datos de ubicación

+ datos de organización

Problemas:

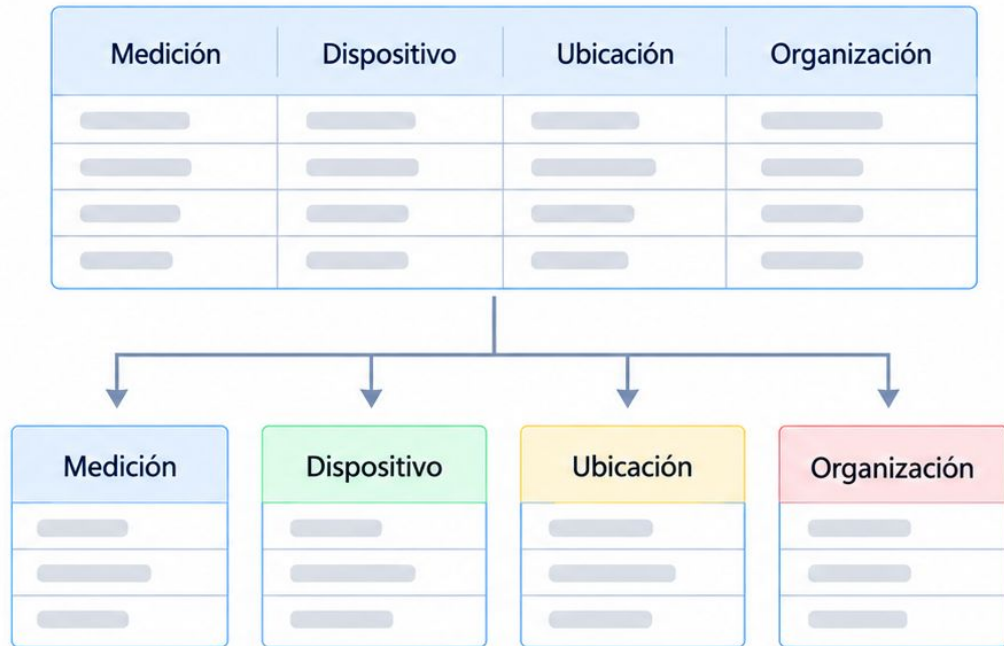
- información repetida;
- múltiples actualizaciones;
- posibles inconsistencias;
- dependencias mezcladas.



La normalización nos ayuda a reorganizar esas dependencias de manera más consistente.

2

Normalización



¿Qué es normalizar?

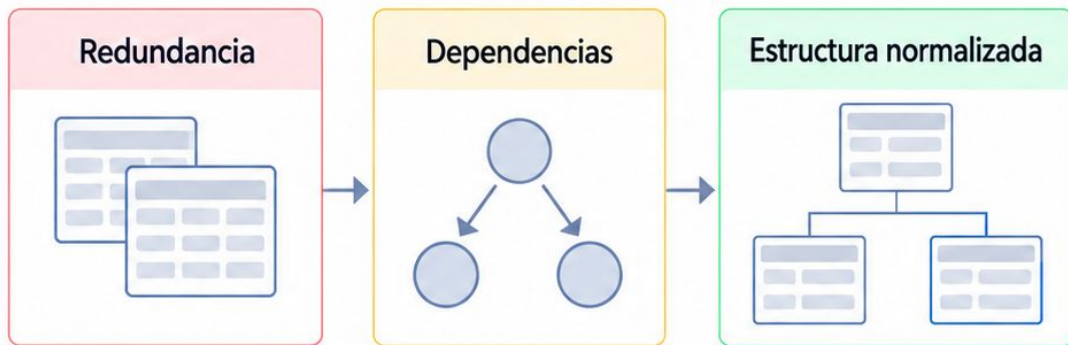
Organizar los datos según sus dependencias

Normalizar busca:

- reducir redundancia innecesaria;
- evitar dependencias problemáticas;
- mantener cada dato donde corresponde;
- disminuir inconsistencias.

Veremos:

- Primera Forma Normal;
- Segunda Forma Normal;
- Tercera Forma Normal.



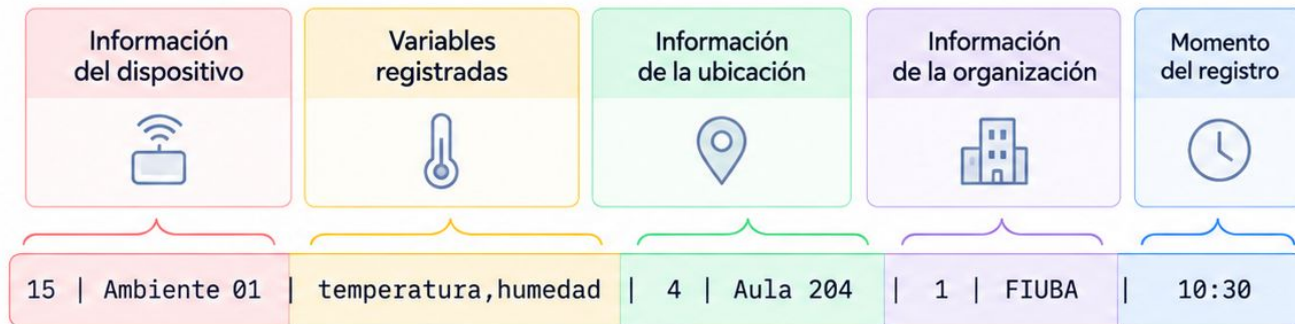
Normalizar no significa crear muchas tablas. Significa organizar correctamente las dependencias.

Nuestro punto de partida

Una estructura que necesita revisión

registro_ambiental

dispositivo_id
dispositivo_nombre
variables
ubicacion_id
ubicacion_nombre
organizacion_id
organizacion_nombre
timestamp



Ejemplo:

15 | Ambiente 01 | temperatura,humedad | 4 | Aula 204 | 1 | FIUBA | 10:30

Encontramos valores múltiples y datos repetidos.



Vamos a mejorar la estructura progresivamente.

Primera Forma Normal

Valores individuales

Problemático:

`variables = "temperatura, humedad"`

Podemos representar:

15 | temperatura | 24.6 | 10:30

15 | humedad | 58 | 10:30

Registro original (con múltiples valores)		
dispositivo_id	variables	momento
15	temperatura, humedad	10:30



Registros en Primera Forma Normal (valores individuales)			
dispositivo_id	variable	valor	momento
15	temperatura	24.6	10:30
15	humedad	58	10:30

Cada fila representa una medición determinada.



En 1FN evitamos almacenar grupos o listas de valores dentro de una misma celda.

Segunda Forma Normal

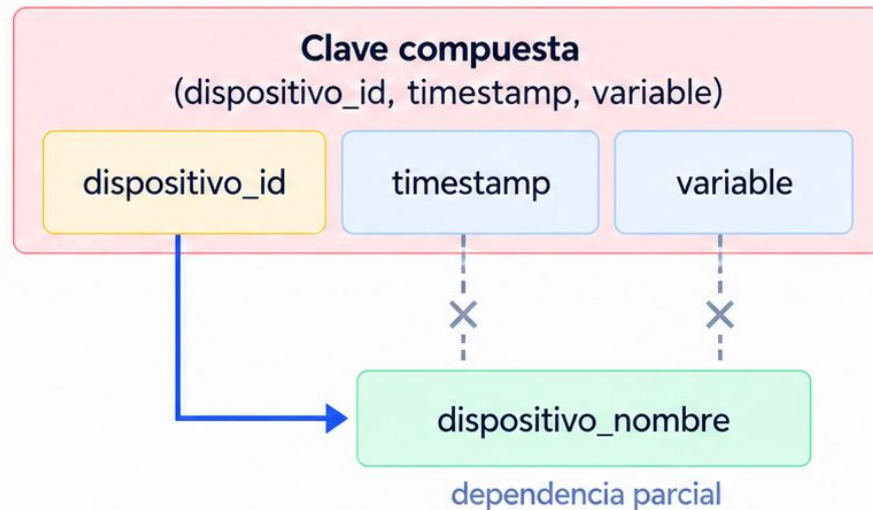
Evitar dependencias parciales

Supongamos una clave formada por:
(dispositivo_id, timestamp, variable)

y almacenamos:
dispositivo_nombre

Pero el nombre depende solamente de:
dispositivo_id

No depende de la clave completa.



En 2FN, los atributos deben depender de la clave completa cuando utilizamos una clave compuesta.

Resolver la dependencia parcial

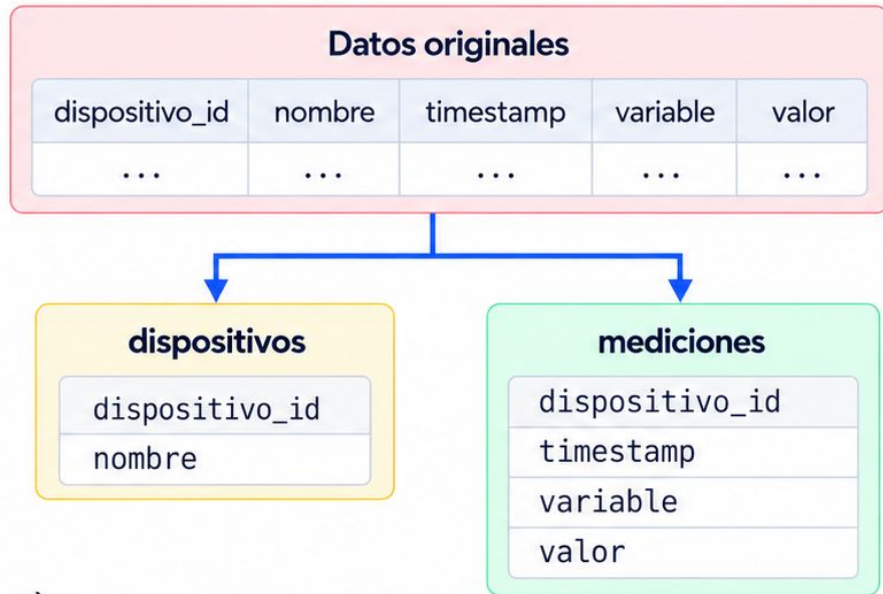
Cada dato en su estructura

dispositivos

dispositivo_id
nombre

mediciones

dispositivo_id
timestamp
variable
valor



El nombre se almacena una sola vez en `dispositivos`.



El nombre describe al dispositivo, no a cada una de sus mediciones.

Tercera Forma Normal

Evitar dependencias transitivas

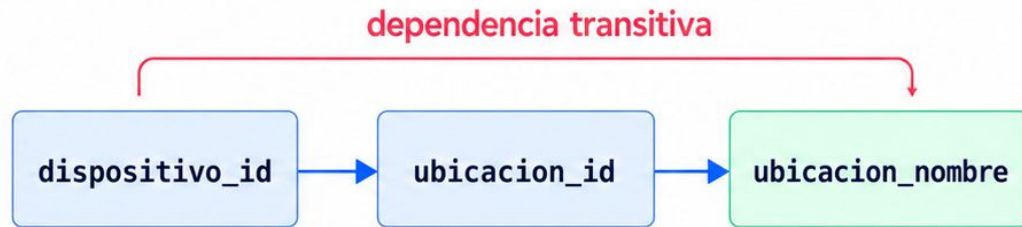
Supongamos:

dispositivos
dispositivo_id
ubicacion_id
ubicacion_nombre

Tenemos:

dispositivo_id → ubicacion_id
ubicacion_id → ubicacion_nombre

ubicacion_nombre depende realmente de la ubicación.



En 3FN evitamos que atributos no clave dependan de otros atributos no clave.

Resolver la dependencia transitiva

Mantener la relación sin repetir los datos

dispositivos

dispositivos
dispositivo_id
ubicacion_id
nombre
numero_serie

ubicaciones

ubicaciones
ubicacion_id
nombre

dispositivos
dispositivo_id
ubicacion_id
nombre
numero_serie



ubicaciones
ubicacion_id
nombre

El nombre de la ubicación se almacena solo una vez en esta tabla.

La relación continúa existiendo mediante `ubicacion\_id`.



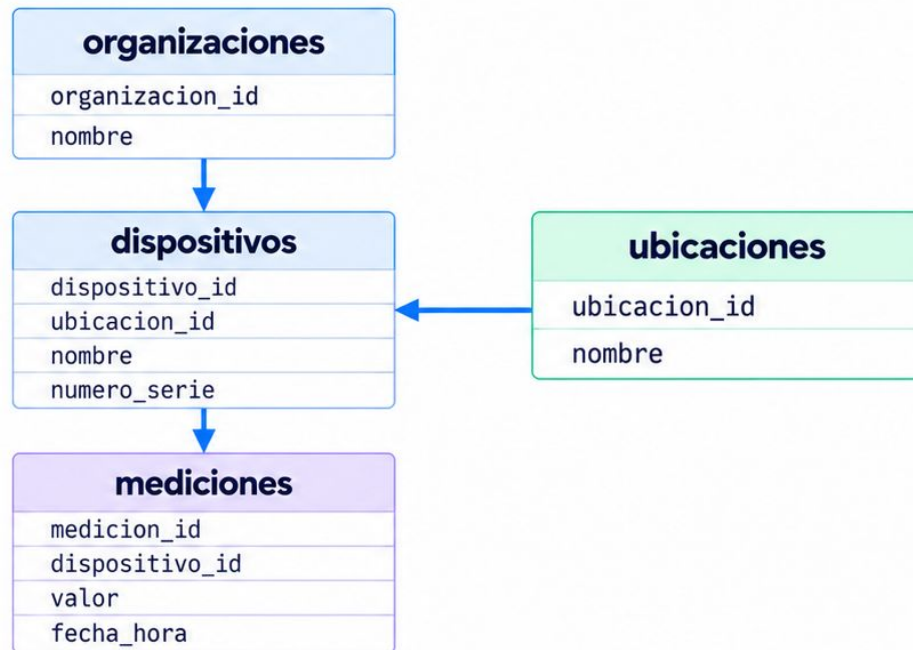
Relacionar tablas permite reducir repetición sin perder el contexto.

La estructura resulta familiar

Ahora entendemos por qué estaba separada

Cada tabla contiene información correspondiente a un concepto.

Los JOIN permiten reconstruir el contexto cuando lo necesitamos.



La normalización explica decisiones que ya utilizamos durante la Clase 3.

1FN, 2FN y 3FN

Tres problemas diferentes

1FN

Evitar listas o grupos dentro de una celda.

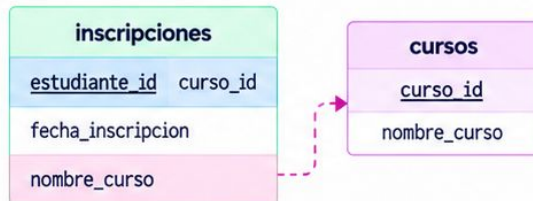
estudiante_id	cursos
1	Matemática, Física
2	Historia, Arte



estudiante_id	curso
1	Matemática
1	Física
2	Historia
2	Arte

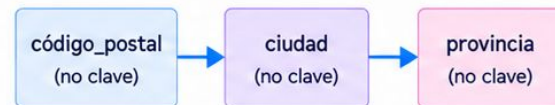
2FN

Evitar dependencias de solamente una parte de una clave compuesta.



3FN

Evitar dependencias entre atributos no clave.



Las formas normales ayudan a detectar distintos problemas de organización.

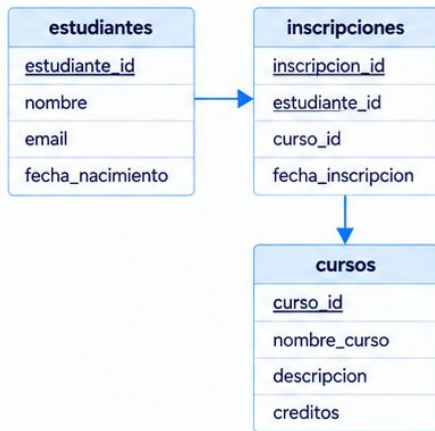
Un criterio, no una competencia

Más tablas no significa automáticamente mejor diseño

Normalizar no significa:

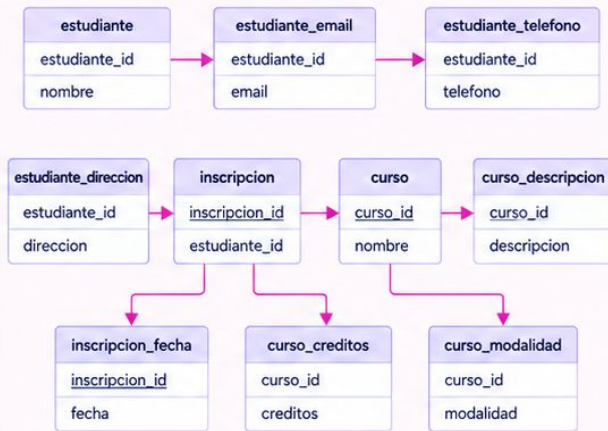
- crear una tabla por atributo;
- separar información sin necesidad;
- aplicar reglas mecánicamente;
- aumentar complejidad sin beneficio.

Modelo equilibrado



VS

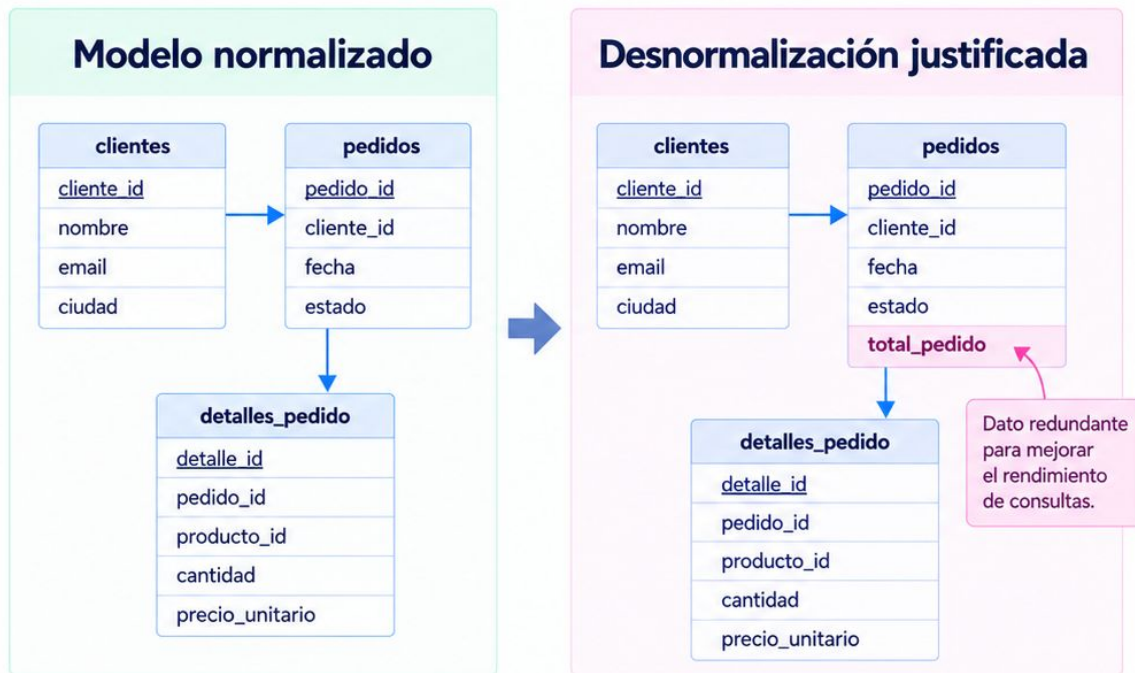
Fragmentación excesiva



Normalizamos para resolver problemas de diseño, no para producir la mayor cantidad posible de tablas.

3

Desnormalización Justificada



Normalización y rendimiento

El diseño también depende del uso

Normalizar favorece:

- consistencia;
- mantenimiento;
- menor redundancia.

Pero también debemos considerar:

- patrones de consulta;
- frecuencia de acceso;
- costo de operaciones;
- requerimientos de rendimiento.

En algunos casos puede analizarse una redundancia controlada.



Desnormalizar debe responder a un requerimiento concreto.

Evitar un cálculo repetido

Redundancia deliberada

Podemos calcular siempre:

```
SELECT dispositivo_id, COUNT(*)  
FROM mediciones  
GROUP BY dispositivo_id;
```

O mantener un contador previamente actualizado.

Sin redundancia

Consulta más costosa.

Con redundancia

Lectura más simple, pero las escrituras deben mantener el contador.

Lectura vs escritura

Sin redundancia



Tabla mediciones
(muchas filas)



Consulta con
**COUNT() +
GROUP BY**



Total por
dispositivo

Lectura más costosa

Con redundancia



Tabla dispositivos
(con total_mediciones)



Lectura directa del
contador



Al insertar una nueva medición
también se debe actualizar
el contador.

Lectura más simple y rápida



Optimizar una operación puede trasladar el costo hacia otra.

Desnormalizar con una razón

No por comodidad

Puede analizarse cuando existe:

- una consulta muy frecuente;
- un cálculo costoso repetitivo;
- información agregada reutilizada constantemente;
- un requerimiento de latencia;
- una necesidad comprobada mediante mediciones.

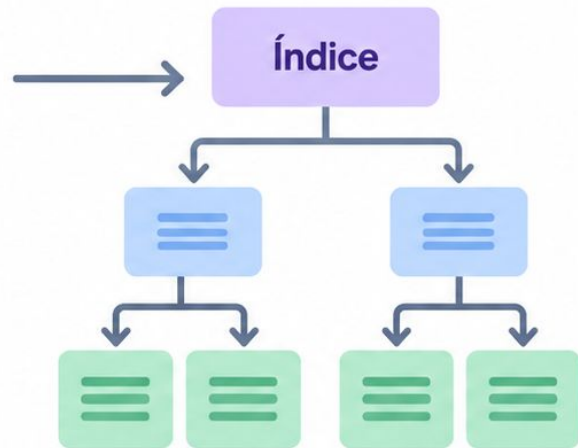


Primero identificamos y medimos el problema. Después evaluamos la desnormalización.

4

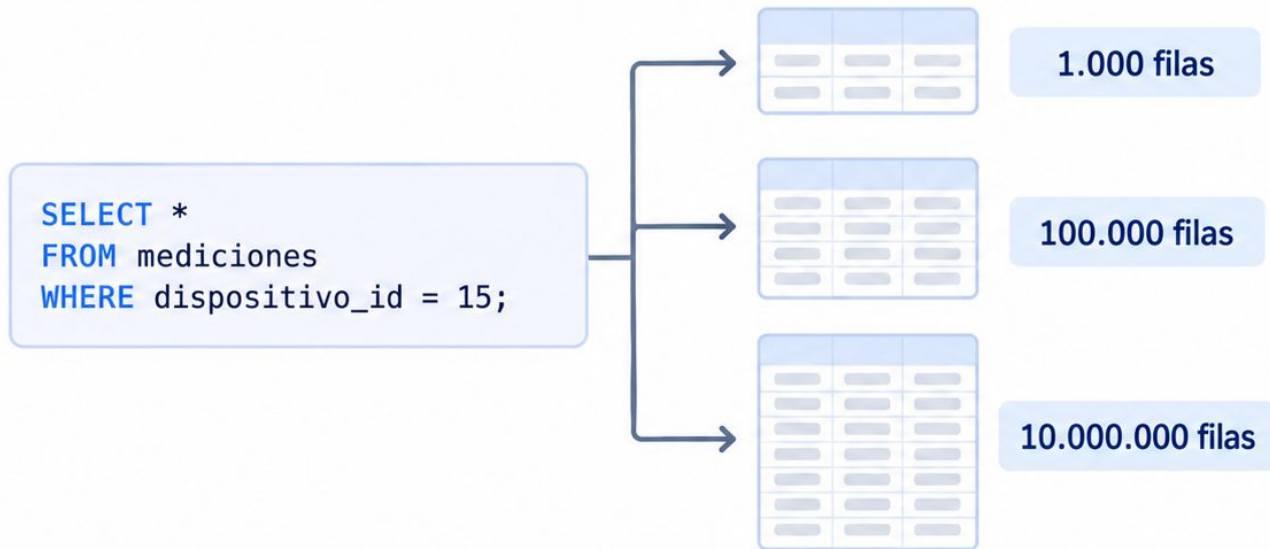
Índices

id	nombre	fecha	monto



La consulta es la misma

La cantidad de filas no



La consulta no cambió.
El volumen de datos sí.



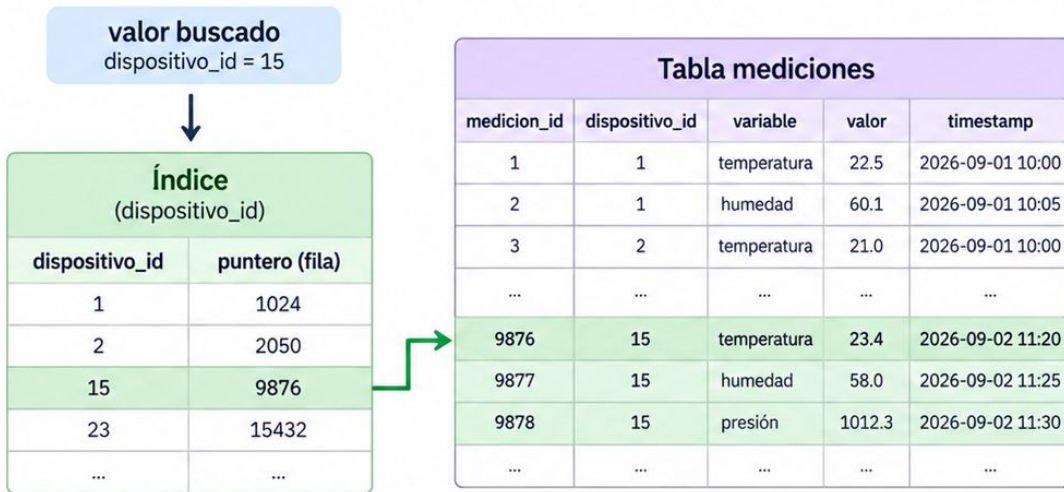
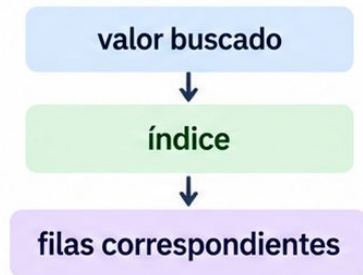
Una consulta correcta puede volverse costosa cuando crece la tabla.

¿Qué es un índice?

Una estructura auxiliar de acceso

Un índice ayuda al SGBD a localizar determinadas filas sin necesariamente recorrer toda la tabla.

Conceptualmente:



El índice no reemplaza la tabla.



Un índice agrega otra forma de encontrar la información.

Índice simple

Indexar una columna

```
CREATE INDEX idx_mediciones_dispositivo  
ON mediciones (dispositivo_id);
```

Puede resultar interesante para consultas frecuentes como:

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = 15;
```

WHERE dispositivo_id = 15

Índice (idx_mediciones_dispositivo)	
dispositivo_id	puntero (fila)
1	1024
2	2050
15	9876
23	15432
...	...

Tabla mediciones				
medicion_id	dispositivo_id	variable	valor	timestamp
9874	8	temperatura	21.3	2026-09-02 10:15
9875	3	humedad	45.2	2026-09-02 10:17
9876	15	temperatura	23.4	2026-09-02 11:20
9877	15	humedad	58.0	2026-09-02 11:25
9878	15	presión	1012.3	2026-09-02 11:30
9879	4	temperatura	20.1	2026-09-02 11:35
...	...	...	...	...



El diseño del índice parte de cómo consultamos los datos.

Patrón de consulta

Dispositivo + período

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = 15  
      AND timestamp >= '2026-09-01'  
      AND timestamp < '2026-09-02'  
ORDER BY timestamp;
```

Utilizamos:

dispositivo_id



timestamp



dispositivo_id = 15



Período de tiempo

Desde: 2026-09-01

Hasta: 2026-09-02



Resultados de la consulta

medicion_id	dispositivo_id	variable	valor	timestamp
9876	15	temperatura	23.4	2026-09-01 00:12
9877	15	humedad	58.0	2026-09-01 01:05
9878	15	presión	1012.3	2026-09-01 03:20
9880	15	temperatura	22.1	2026-09-01 10:17
9882	15	humedad	46.7	2026-09-01 18:45
...	...	...	...	...

Filtrado por dispositivo y período, ordenado por timestamp.



Los patrones reales de consulta indican qué combinaciones conviene analizar.

Índice compuesto

Representar un patrón de acceso

```
CREATE INDEX idx_mediciones_dispositivo_timestamp  
ON mediciones (dispositivo_id, timestamp);
```

Puede resultar adecuado para:

un dispositivo determinado

+

un rango de tiempo

Índice compuesto
(dispositivo_id, timestamp)

dispositivo_id	timestamp
8	2026-09-01 08:15
8	2026-09-01 11:20
8	2026-09-02 09:05
...	...
15	2026-09-01 00:12
15	2026-09-01 10:17
15	2026-09-01 18:45
15	2026-09-02 07:30
...	...
23	2026-09-01 03:21
23	2026-09-01 16:10
...	...

primero por
dispositivo

luego por
timestamp
(en orden ascendente)



Un índice compuesto puede representar mejor consultas que utilizan varias columnas.

Orden de las columnas

Dos índices pueden organizarse diferente

(dispositivo_id, timestamp)

Conceptualmente:

Dispositivo 1

- 10:00
- 10:05
- 10:10

Dispositivo 2

- 10:00
- 10:05

No representa exactamente lo mismo que:

(timestamp, dispositivo_id)

(dispositivo_id, timestamp)

Primero por dispositivo,
luego por timestamp

Dispositivo 1

- 10:00
- 10:05
- 10:10

Dispositivo 2

- 10:00
- 10:05

VS

(timestamp, dispositivo_id)

Primero por timestamp,
luego por dispositivo

10:00

- Dispositivo 1
- Dispositivo 2

10:05

- Dispositivo 1
- Dispositivo 2

10:10

- Dispositivo 1



El orden debe analizarse según el patrón de consulta.

Selectividad

No todos los filtros reducen igual

Sobre un millón de filas:

WHERE medicion_id = 824315

puede devolver:

1 fila.

Mientras:

WHERE variable = 'temperatura'

puede devolver
cientos de miles.

Filtro muy selectivo

medicacion_id = 824315

medicacion_id	dispositivo_id	variable	valor	timestamp

... 1.000.000 de filas ...

Filtro poco selectivo

variable = 'temperatura'

medicion_id	dispositivo_id	variable	valor	timestamp
		temperatura		
		temperatura		
		temperatura		
		temperatura		
		temperatura		
		temperatura		
		temperatura		

... 1.000.000 de filas ...



La selectividad indica cuánto permite distinguir una condición dentro del conjunto de datos.

La utilidad potencial de un índice

```
numero_serie = 'AMB-00482'
```

Pocas filas.

Menor selectividad

```
activo = true
```

Quizás casi todas las filas.

Mayor selectividad

```
numero_serie = 'AMB-00482'
```

id	numero_serie	activo	ubicacion	timestamp
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]

Pocas filas →

... 1.000.000 de filas ...

Menor selectividad

`activo = true`

id	numero_serie	activo	ubicacion	timestamp
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]

Casi toda la tabla

... 1.000.000 de filas ...



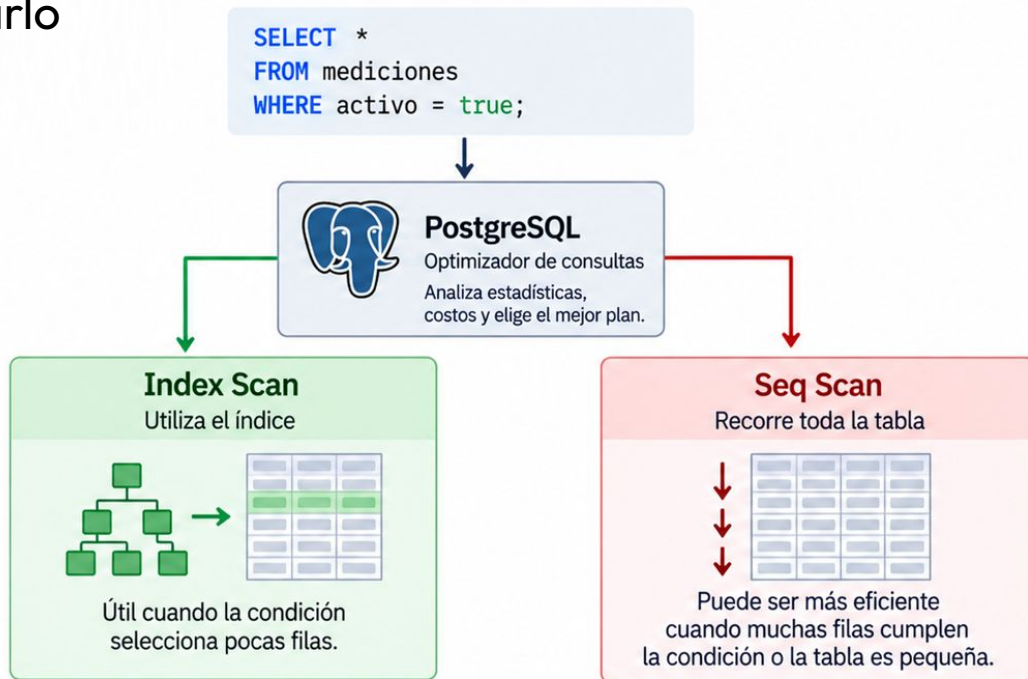
Un índice suele resultar más interesante cuando permite localizar una parte pequeña de la tabla.

El optimizador decide

Crear un índice no obliga a utilizarlo

Aunque exista un índice, PostgreSQL puede preferir recorrer la tabla si:

- la tabla es pequeña;
- muchas filas cumplen la condición;
- el índice aporta poco beneficio.



Que exista un índice no significa que PostgreSQL deba utilizarlo.

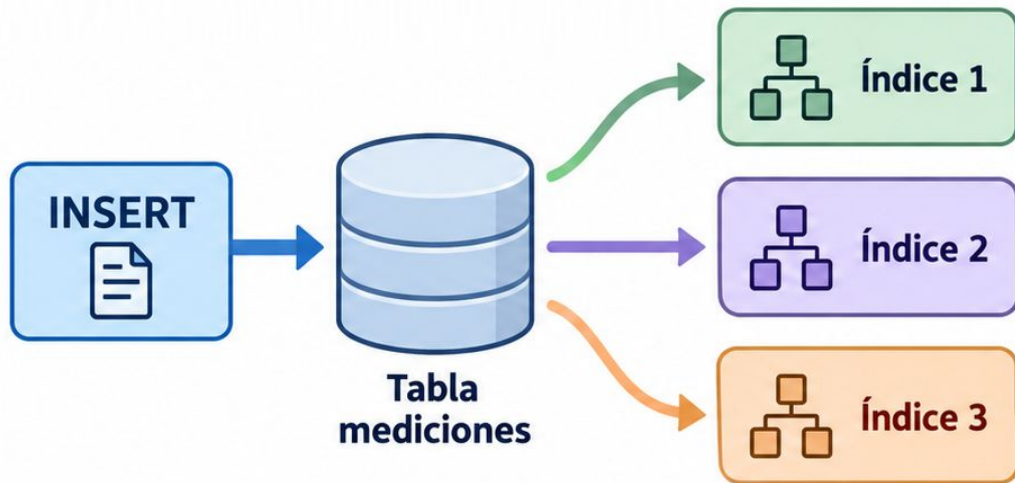
Los índices tienen costo

Mantener estructuras adicionales

Cada índice:

- ocupa almacenamiento;
- debe actualizarse con INSERT;
- puede cambiar con UPDATE;
- debe mantenerse con DELETE.

Cuanto más índices existan, mayor puede ser el costo de las escrituras.



más índices = más trabajo en escritura

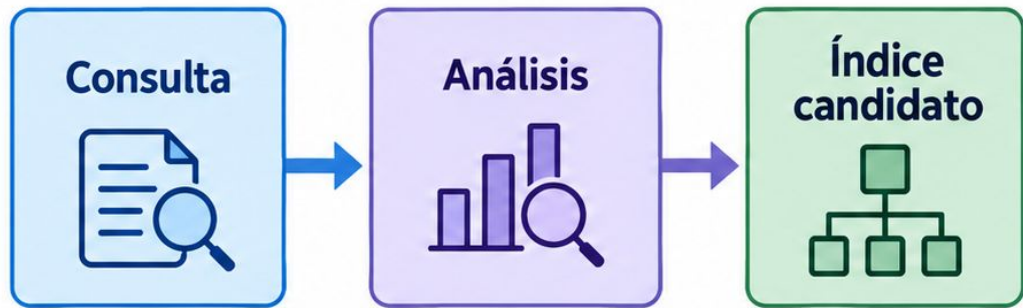


Un índice es una estructura real, no una optimización gratuita.

Un índice necesita una razón

Diseñar desde el uso

- ¿Qué consulta queremos mejorar?
- ¿Qué columnas utiliza?
- ¿Cuántas filas devuelve?
- ¿Con qué frecuencia se ejecuta?
- ¿La tabla recibe muchas escrituras?
- ¿Existe ya un índice similar?



Primero identificamos el patrón de acceso. Después diseñamos y evaluamos el índice.

5

Planes de Ejecución



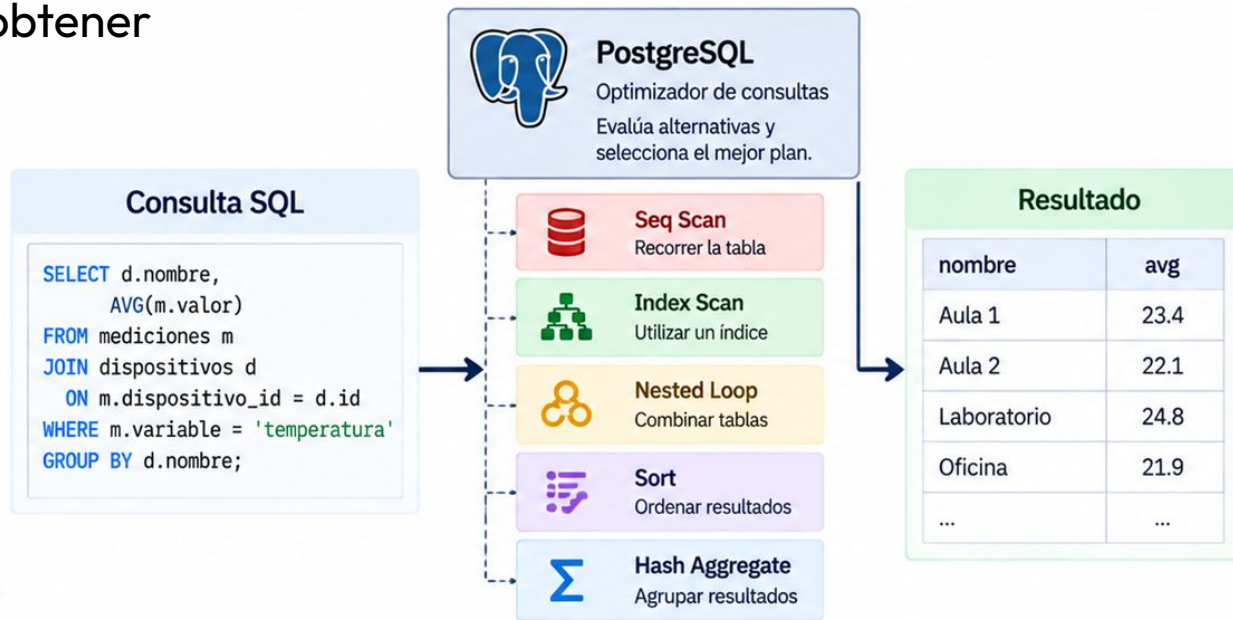
Una consulta, varias estrategias

SQL indica qué queremos obtener

Ante una consulta PostgreSQL puede evaluar alternativas como:

- recorrer una tabla;
- utilizar un índice;
- combinar tablas;
- ordenar;
- agrupar.

El SGBD selecciona una estrategia.



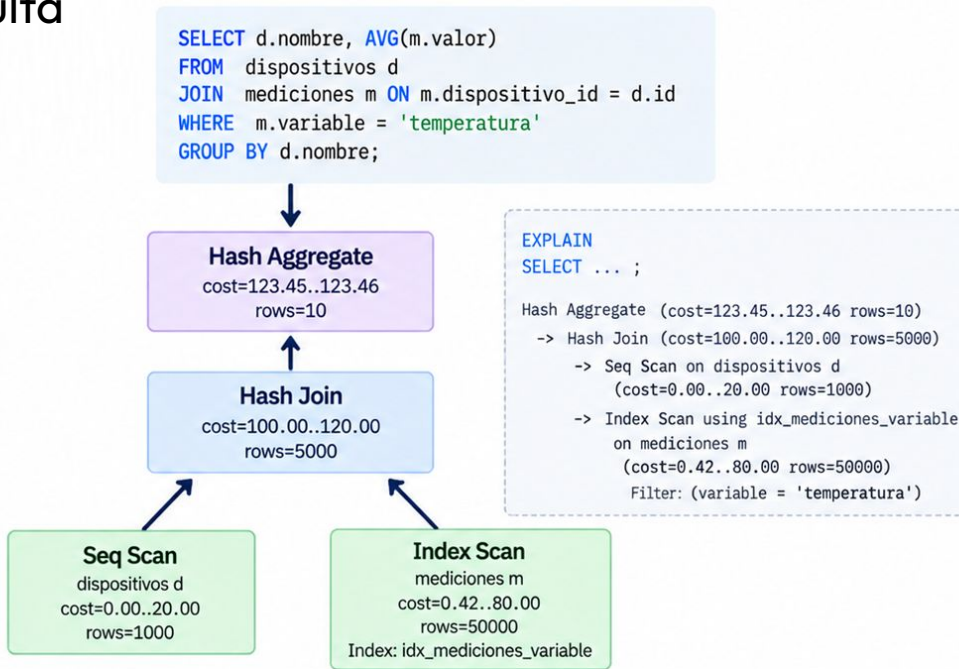
La consulta expresa qué queremos. El plan describe cómo PostgreSQL intenta obtenerlo.

Plan de ejecución

La estrategia para resolver la consulta

Un plan permite observar:

- cómo se acceden las tablas;
- si aparece un índice;
- cómo se combinan operaciones;
- cuántas filas se estiman;
- qué costo estima PostgreSQL.



El plan permite observar cómo piensa resolver PostgreSQL una consulta.

EXPLAIN

Observar el plan estimado

PostgreSQL devuelve el plan que utilizaría para resolver la consulta.

Consulta SQL

```
EXPLAIN
SELECT *
FROM mediciones
WHERE dispositivo_id = 15;
```



Plan estimado (salida simplificada)

Index Scan using idx_mediciones_dispositivo

on mediciones (cost=0.42..8.44 rows=120 width=56)

Index Cond: (dispositivo_id = 15)

Tipo de operación
y índice utilizado

Costo estimado
y filas estimadas

Condición aplicada



EXPLAIN permite observar la estrategia estimada sin centrarnos todavía en la ejecución real.

Dos formas de acceso

Recorrer la tabla o utilizar un índice

Seq Scan

Tabla → revisar filas

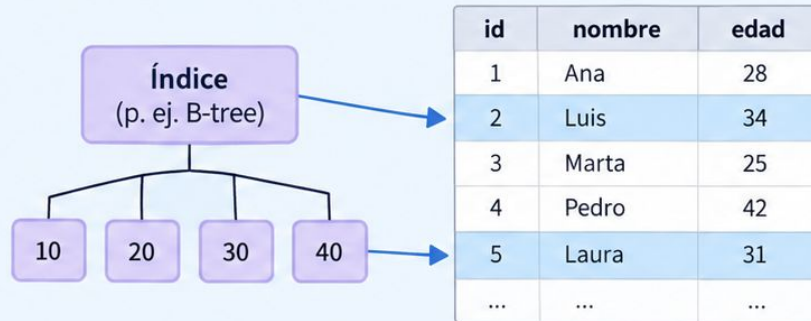


id	nombre	edad
1	Ana	28
2	Luis	34
3	Marta	25
4	Pedro	42
5	Laura	31
...	...	...

Se revisan todas las filas de la tabla, una por una.

Index Scan

Índice → localizar filas



Un 'Seq Scan' puede ser perfectamente válido cuando la tabla es pequeña o cuando muchas filas cumplen la condición.



Un recorrido secuencial no es automáticamente una mala decisión.

Estimaciones del optimizador

Comparar alternativas

Podemos encontrar:

`cost=...`

`rows=...`

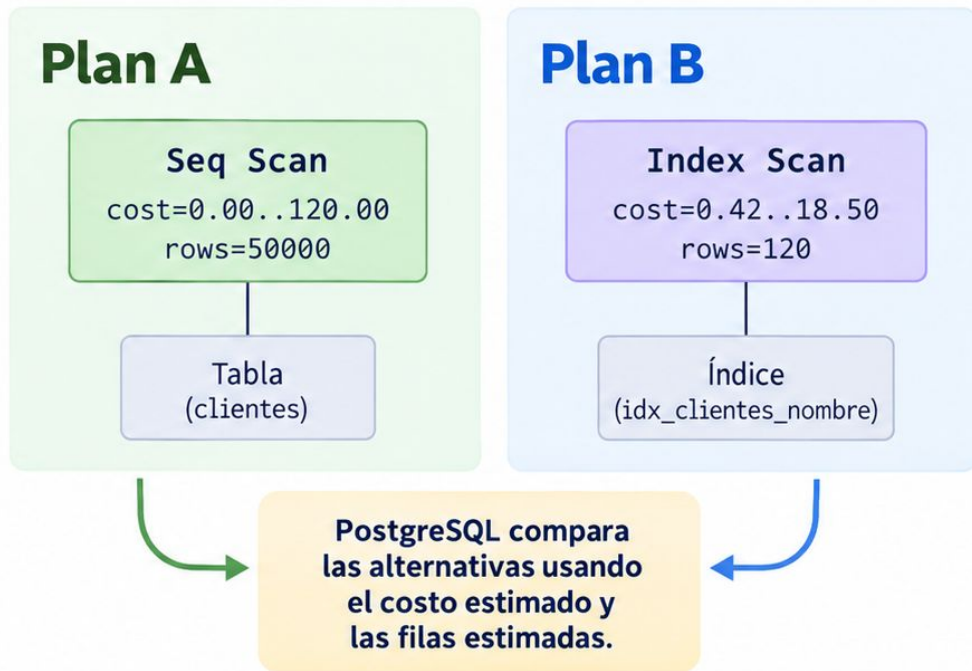
`cost`

Representa una estimación utilizada para comparar estrategias.

`rows`

Representa cuántas filas espera encontrar PostgreSQL.

Los valores de ``cost`` no equivalen directamente a segundos.



Las estimaciones ayudan al optimizador a elegir el plan que considera más conveniente.

EXPLAIN ANALYZE

Ejecutar y observar qué ocurrió

EXPLAIN ANALYZE

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = 15;
```

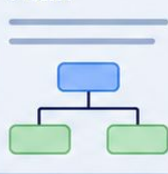
La consulta se ejecuta.

Podemos comparar:

- estimaciones;
- filas reales;
- tiempos observados.

EXPLAIN

Plan



Plan estimado

Estimaciones de costos
y filas

EXPLAIN ANALYZE



Ejecución real

Filas reales
Tiempos observados
(ej. tiempo total, por nodo)

Permiten comparar
el plan estimado con la
ejecución real.



EXPLAIN estima. EXPLAIN ANALYZE ejecuta y permite observar lo ocurrido.

Medir antes de cambiar

Necesitamos una referencia

Podríamos encontrar:

Seq Scan on mediciones

Registramos ese comportamiento antes de modificar nada.

Consulta SQL

```
EXPLAIN ANALYZE
SELECT *
FROM mediciones
WHERE dispositivo_id = 15
      AND timestamp >= '2026-09-01'
      AND timestamp < '2026-09-02';
```

Plan obtenido (antes)

ANTES

```
Seq Scan on mediciones (cost=0.00..4312.50 rows=12500 width=56)
  (actual time=0.021..125.487 rows=12340 loops=1)
  Filter: (((dispositivo_id = 15) AND ("timestamp" >= '2026-09-01'::timestamp
    AND "timestamp" < '2026-09-02'::timestamp)))
  Rows Removed by Filter: 987660
  Planning Time: 0.412 ms
  Execution Time: 126.318 ms
```



Antes de optimizar necesitamos conocer el comportamiento inicial.

Índice candidato

Una hipótesis de mejora

Consulta SQL

```
CREATE INDEX idx_mediciones_dispositivo_timestamp  
ON mediciones (dispositivo_id, timestamp);
```

La consulta utiliza ambas columnas.

Ahora PostgreSQL dispone de una nueva alternativa de acceso.

Diagrama conceptual



El índice es una hipótesis de mejora que debemos comprobar.

Volver a medir

Comparar el comportamiento

Ejecutamos nuevamente:

```
EXPLAIN ANALYZE  
SELECT ...
```

Podríamos comparar:

ANTES

Seq Scan

DESPUÉS

Index Scan

También observamos filas y tiempos.

ANTES
(sin índice)

Seq Scan
on usuarios

Filter
(edad > 30)

Filas estimadas:	100,000
Filas reales:	98,532
Tiempo de ejecución:	520.432 ms

Después
de agregar
un índice

DESPUÉS
(con índice)

Index Scan
on usuarios
using idx_usuarios_edad

Filter
(edad > 30)

Filas estimadas:	1,200
Filas reales:	1,034
Tiempo de ejecución:	12.847 ms



Una mejora debe verificarse comparando antes y después.

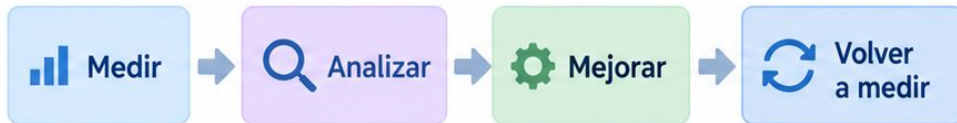
¿Y si continúa con Seq Scan?

También puede ser una decisión válida

Puede ocurrir porque:

- la tabla es pequeña;
- muchas filas cumplen el filtro;
- el índice aporta poco;
- recorrer la tabla resulta más barato.

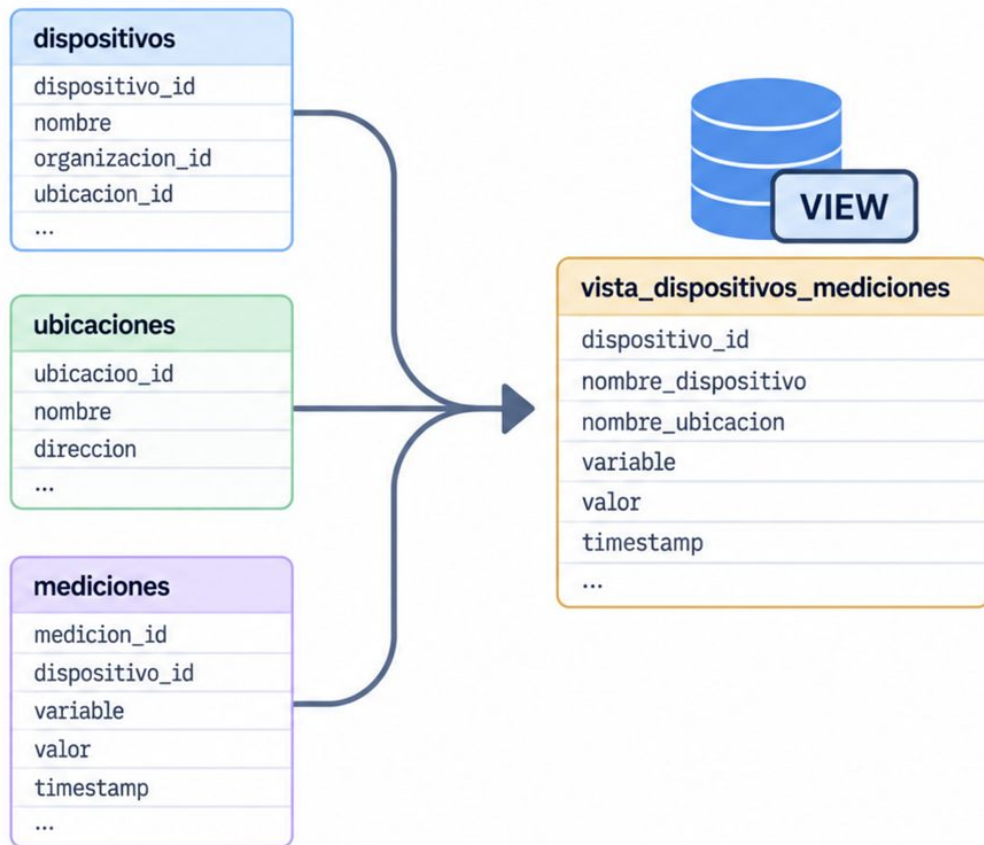
Proceso de optimización:



El objetivo no es conseguir un Index Scan. Es comprender y mejorar el comportamiento real.

6

Vistas y Vistas Materializadas



Una consulta compleja

Recuperar el contexto completo

Para mostrar una medición podemos necesitar combinar:



mediciones
medicion_id
dispositivo_id
variable
valor
timestamp
...

dispositivos
dispositivo_id
organizacion_id
ubicacion_id
nombre
numero_serie
...

ubicaciones
ubicacion_id
organizacion_id
nombre
direccion
...

organizaciones
organizacion_id
nombre
cuit
...

Resultado de la consulta					
timestamp	variable	valor	dispositivo	ubicación	organización
2026-09-01 10:15	temperatura	24.8	Sensor Aula 1	Aula 1	Escuela Norte
2026-09-01 10:20	humedad	56.2	Sensor Lab	Laboratorio	Escuela Norte
2026-09-01 10:25	presión	1013.2	Sensor Aula 2	Aula 2	Escuela Sur
...	...	...	...	...	...

La misma consulta puede aparecer en diferentes partes de una aplicación.



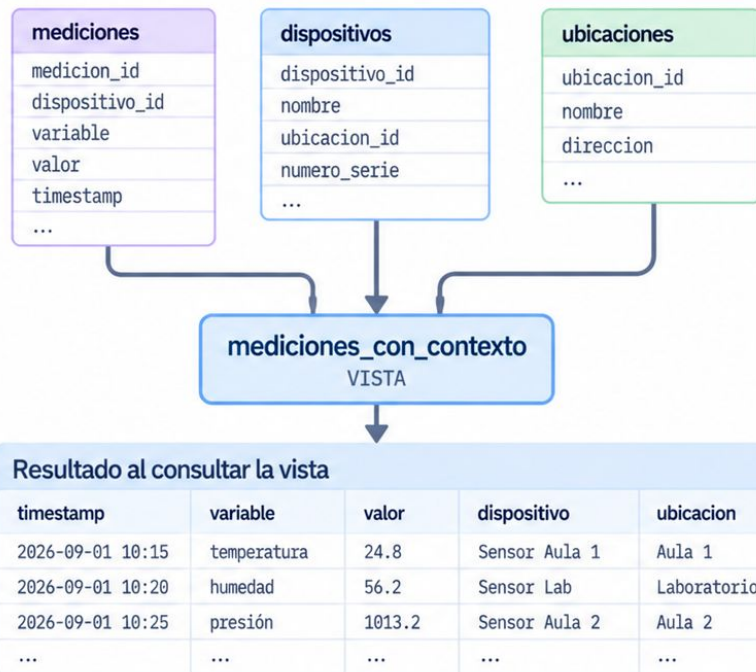
A veces queremos reutilizar una consulta como una estructura conocida.

Vista

Dar un nombre a una consulta

Definición de la vista

```
CREATE VIEW mediciones_con_contexto AS
SELECT
    m.timestamp,
    m.variable,
    m.valor,
    d.nombre AS dispositivo,
    u.nombre AS ubicacion
FROM mediciones m
JOIN dispositivos d
    ON m.dispositivo_id = d.dispositivo_id
LEFT JOIN ubicaciones u
    ON d.ubicacion_id = u.ubicacion_id;
```



Una vista encapsula una consulta para poder reutilizarla.

Una interfaz más simple

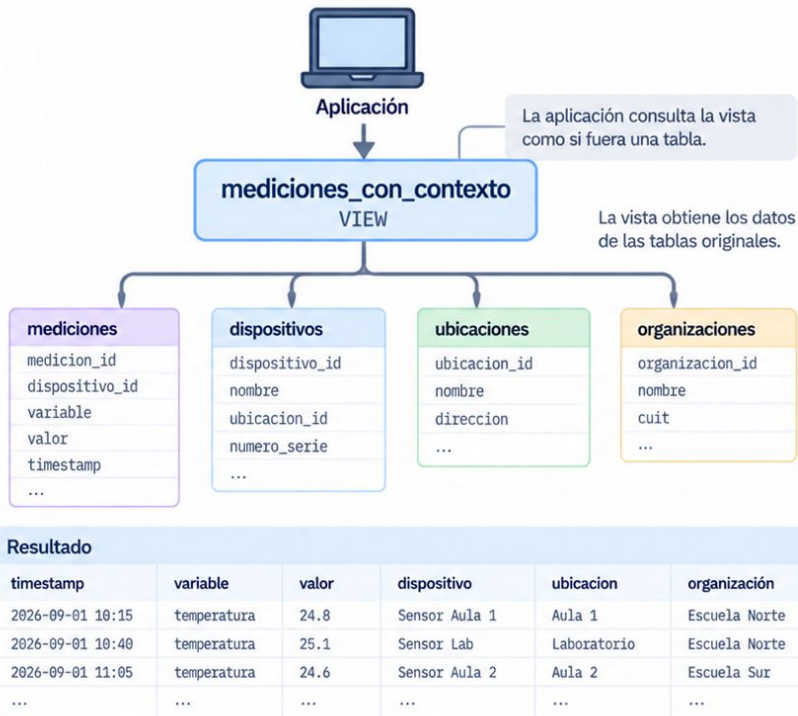
Los datos siguen en las tablas originales

Consulta a la vista

```
SELECT *  
FROM mediciones_con_contexto  
WHERE variable = 'temperatura';
```

La vista convencional almacena la definición de la consulta.

No crea una copia independiente de todas las filas.



Una VIEW simplifica el acceso sin duplicar físicamente el resultado.

Vista materializada

Guardar el resultado de una consulta

Definición de la vista materializada

```
CREATE MATERIALIZED VIEW resumen_mediciones AS
SELECT
    dispositivo_id,
    variable,
    AVG(valor) AS promedio
FROM mediciones
GROUP BY dispositivo_id, variable;
```

El resultado queda almacenado.

Puede ser útil cuando calcularlo es costoso
y se consulta frecuentemente.



Una vista materializada almacena el resultado previamente calculado.

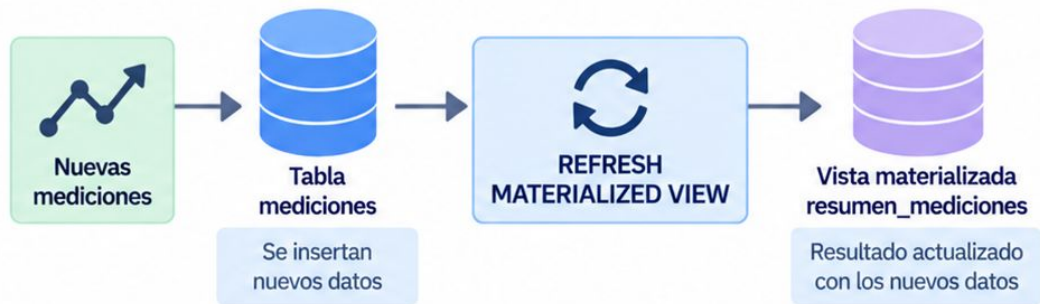
Actualizar el resultado

Los datos pueden quedar desactualizados

Ingresan nuevas mediciones, pero la vista materializada conserva el resultado anterior hasta actualizarla.

Comando

```
REFRESH MATERIALIZED VIEW  
resumen_mediciones;
```



Antes de actualizar

dispositivo_id	variable	promedio
1	temperatura	24.8
1	humedad	56.1
2	temperatura	25.3
2	presión	1012.4
...	...	...



Después de actualizar

dispositivo_id	variable	promedio
1	temperatura	25.1
1	humedad	55.8
2	temperatura	24.9
2	presión	1011.7
...	...	...



Guardar un resultado implica decidir cuándo debe volver a calcularse.

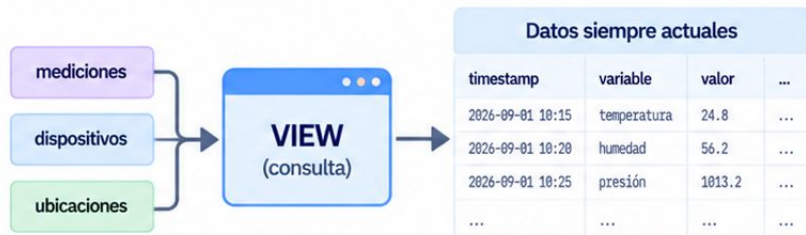
Dos necesidades diferentes

Consulta actual o resultado almacenado

VIEW

Almacena la consulta y muestra datos actuales

- almacena la consulta;
- muestra datos actuales;
- simplifica el acceso.

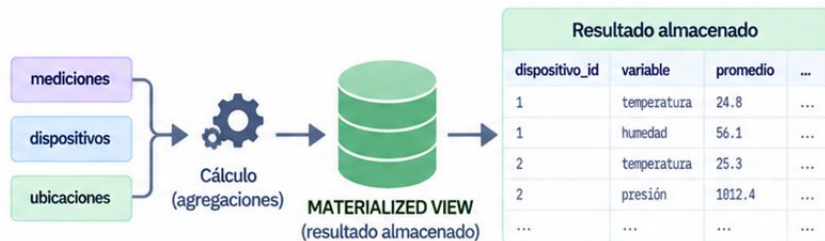


Cada consulta ejecuta nuevamente la consulta sobre las tablas originales.

MATERIALIZED VIEW

Almacena el resultado y puede acelerar consultas

- almacena el resultado;
- puede acelerar cálculos costosos;
- necesita actualización.



El resultado queda guardado.
Debe actualizarse cuando sea necesario.



La elección depende de si necesitamos datos siempre actuales o resultados previamente calculados.

7

JSON y JSONB

Datos variables dentro de PostgreSQL.



PostgreSQL



dispositivos			
id	dispositivo_id	nombre	configuracion_json
1	1	Estación Clima	<pre>{ "sensor": "BME280", "frecuencia": 60, "alerta": true }</pre>
2	2	Cámara IP	<pre>{ "resolucion": "1080p", "fps": 30, "vision_nocturna": true }</pre>
3	3	Sensor Ruido	<pre>{ "umbral_db": 70, "promedio_min": 5, "activo": true }</pre>
...	...	...	...

Estructuras variables

Algunas configuraciones pueden variar

Dispositivo A:

- intervalo
- umbral_temperatura

Dispositivo B:

- intervalo
- modo_ahorro
- presion_habilitada

Agregar una columna para cada posible parámetro puede producir muchas columnas opcionales.

	Dispositivo A Sensor de Temperatura
intervalo	60
umbral_temperatura	30

	Dispositivo B Cámara de Seguridad
intervalo	60
modo_ahorro	true
presion_habilitada	false

id	nombre	intervalo	umbral_temperatura	modo_ahorro	presion_habilitada
1	Dispositivo A	60	30	NULL	NULL
2	Dispositivo B	60	NULL	true	false
...	...	...	...	...	...



Una estructura rígida puede resultar incómoda cuando la variabilidad es parte real del problema.

Información semiestructurada

Una estructura puede variar entre filas



Dispositivo 1

Sensor de Temperatura

```
{
  "intervalo": 30,
  "umbral_temperatura": 28
}
```



Dispositivo 2

Cámara de Seguridad

```
{
  "intervalo": 10,
  "modo": "eco",
  "presion_habilitada": true
}
```

PostgreSQL dispone de tipos ``JSON`` y ``JSONB``.



JSONB permite incorporar información variable dentro de una base relacional.

JSON y JSONB

No son exactamente lo mismo

PostgreSQL ofrece dos tipos para almacenar documentos JSON.

JSON

Guarda el contenido en formato JSON textual.

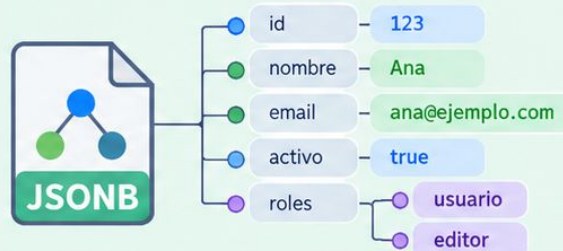


```
1 {  
2   "id": 123,  
3   "nombre": "Ana",  
4   "email": "ana@ejemplo.com",  
5   "activo": true,  
6   "roles": ["usuario", "editor"]  
7 }
```

JSON → documento textual

JSONB

Guarda el contenido en una estructura interna procesada que PostgreSQL puede interpretar.



No transforma el documento en una tabla relacional. Permite trabajar mejor con su estructura interna.

JSONB → estructura interna del documento + búsqueda + índice



Búsqueda

Consultar claves o valores



Filtrado

Filtrar por contenido



Índices

Indexar para mayor rendimiento



Ambos almacenan información en formato JSON, pero **JSONB** está pensado para trabajar mejor con el contenido del documento dentro de PostgreSQL.

Relacional + semiestructurado

Combinar estructura estable y variable

Agregar columna JSONB

```
ALTER TABLE dispositivos  
ADD COLUMN configuracion JSONB;
```

Podemos mantener como columnas:

- dispositivo_id
- organizacion_id
- nombre
- numero_serie

y colocar parámetros variables dentro de **configuracion**.

Tabla dispositivos

dispositivo_id (BIGINT)	organizacion_id (BIGINT)	nombre (TEXT)	numero_serie (TEXT)	configuracion (JSONB)
1	1	Sensor Aula 1	SN-001	{ "intervalo": 30, "umbral_temperatura": 28 }
2	1	Estación Exterior	SN-002	{ "intervalo": 10, "modo": "eco", "presion_habilitada": true }
3	2	Sensor Laboratorio	SN-003	{ "intervalo": 60, "alertas": { "humedad": 70 } }
...	...	...	...	...



No necesariamente debemos elegir entre relacional o JSON: pueden convivir dentro del mismo modelo.

Flexibilidad con criterio

Los datos estables siguen teniendo estructura

Datos como:

- identificador;
- organización;
- número de serie;
- ubicación;

poseen estructura estable, relaciones y restricciones.

No existe una razón clara para moverlos todos dentro de un documento JSON.



Todo en JSONB

Se pierde la estructura y las restricciones.

dispositivo_id	datos (JSONB)
1	{ "organizacion_id": 1, "numero_serie": "SN-001", "ubicacion_id": 10, "intervalo": 30, "umbral_temperatura": 28 }
2	{ "organizacion_id": 2, "numero_serie": "SN-002", "ubicacion_id": 12, "modo": "eco", "presion_habilitada": true }
...	...

Se pierden relaciones, restricciones y la posibilidad de consultar por campos específicos.



Columnas + JSONB

Estructura para lo estable, flexibilidad para lo variable.

dispositivo_id (BIGINT)	organizacion_id (BIGINT)	ubicacion_id (BIGINT)	numero_serie (TEXT)	configuracion (JSONB)
1	1	10	SN-001	{ "intervalo": 30, "umbral_temperatura": 28 }
2	2	12	SN-002	{ "modo": "eco", "presion_habilitada": true }
...	...	...	...	...

Mantenemos el modelo relacional y usamos JSONB para los parámetros variables.



JSONB debería resolver variabilidad real, no reemplazar automáticamente el modelo relacional.

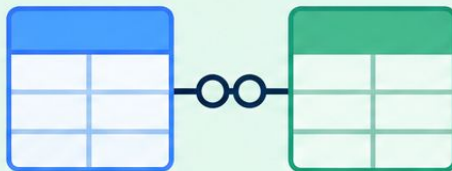
Tres alternativas

¿Qué representa la información?



Columna

Dato estable y conocido.



Relación

Otro concepto con identidad e información propia.



JSONB

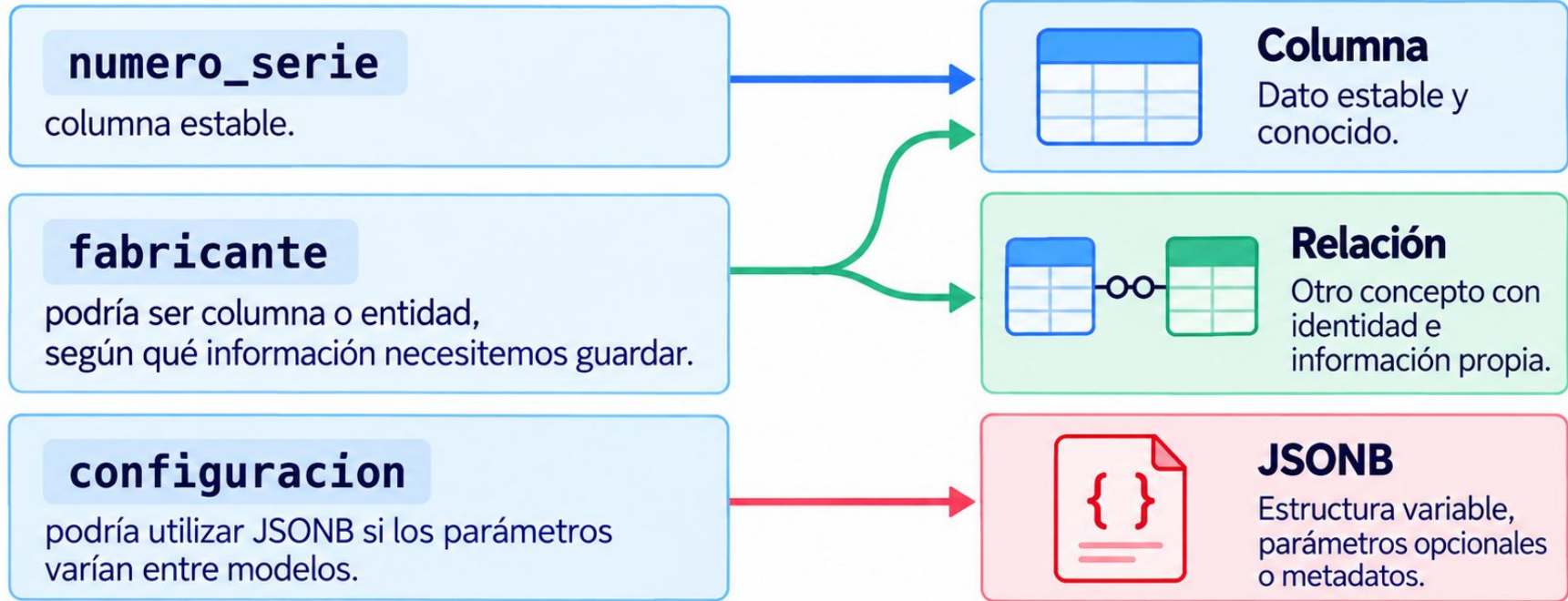
Estructura variable, parámetros opcionales o metadatos.



La representación depende de la naturaleza del dato y de cómo necesitamos utilizarlo.

¿Dónde lo guardaríamos?

Decidir según el requerimiento



No existe una representación correcta sin conocer primero el requerimiento.

JSONB no es una base documental

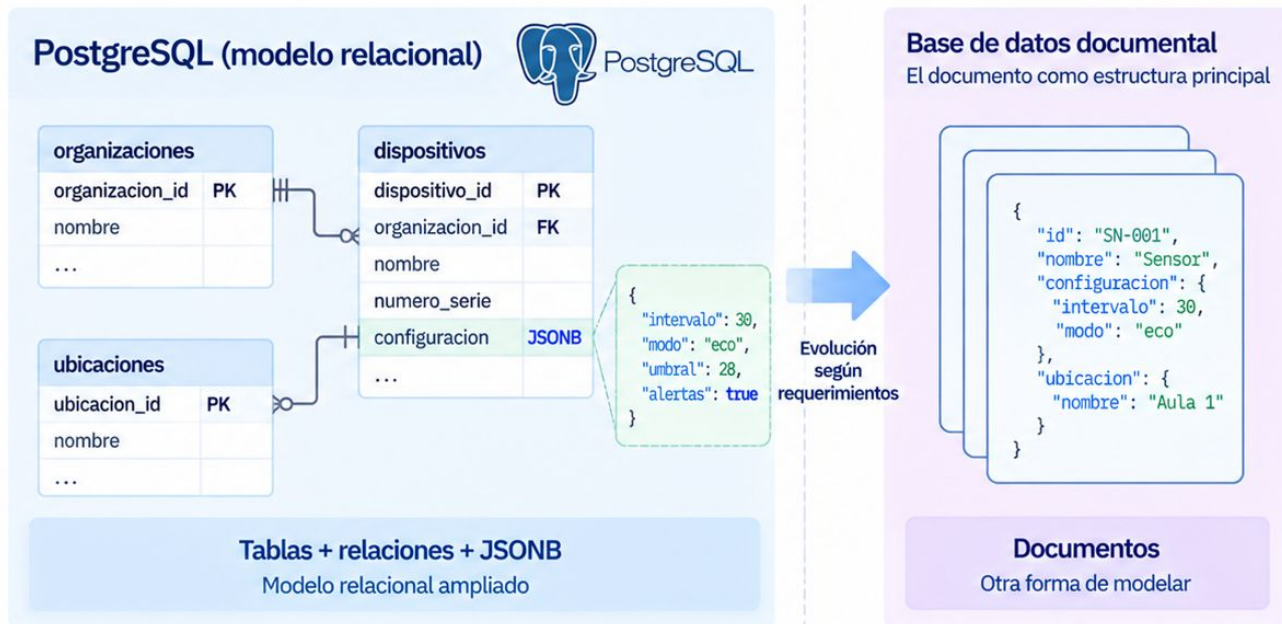
Estamos ampliando PostgreSQL

Seguimos utilizando:

- tablas;
- claves;
- relaciones;
- restricciones;
- SQL.

JSONB permite flexibilizar una parte del esquema.

En la próxima clase veremos modelos donde el documento puede convertirse en la estructura principal.



Utilizar JSONB y utilizar una base documental no son decisiones equivalentes.

8

Migraciones y Evolución del Esquema



El esquema también evoluciona

Una base ya implementada puede necesitar cambios

Nuevo requerimiento:

Necesitamos conocer qué versión de firmware posee cada dispositivo.

Podemos incorporar:

```
ALTER TABLE dispositivos  
ADD COLUMN firmware_version TEXT;
```

Antes

dispositivos

dispositivo_id

organizacion_id

nombre

numero_serie



Después

dispositivos

dispositivo_id

organizacion_id

nombre

numero_serie

firmware_version



Cuando cambia el sistema, también puede necesitar cambiar su estructura de datos.

Cambiar una base en uso

La nueva columna comienza sin información histórica

Las filas existentes todavía no poseen `firmware_version`.

Podemos necesitar:

- permitir NULL temporalmente;
- asignar un valor conocido;
- completar información posteriormente;
- adaptar consultas y aplicaciones.

Después de agregar la columna

Las filas existentes no tienen información.

dispositivo_id	nombre	numero_serie	firmware_version
1	Sensor Aula 1	SN-001	NULL
2	Estación Exterior	SN-002	NULL
3	Sensor Laboratorio	SN-003	NULL
4	Sensor Depósito	SN-004	NULL
5	Estación Patio	SN-005	NULL
...	...	...	...

Completar la información progresivamente

Después de completar los datos

La columna ya contiene la información.

dispositivo_id	nombre	numero_serie	firmware_version
1	Sensor Aula 1	SN-001	1.0.3
2	Estación Exterior	SN-002	2.1.0
3	Sensor Laboratorio	SN-003	1.0.1
4	Sensor Depósito	SN-004	1.2.0
5	Estación Patio	SN-005	2.0.5
...	...	...	...



Una evolución del esquema debe considerar tanto la nueva estructura como los datos existentes.

Cambiar de un esquema a otro

Evolucionar de manera controlada



Una migración puede:

-  **Crear o modificar estructuras**
Tablas, columnas, tipos de datos.
-  **Transformar datos**
Actualizar o migrar información existente.
-  **Incorporar restricciones**
NOT NULL, UNIQUE, CHECK, claves foráneas.
-  **Agregar índices**
Mejorar el rendimiento de las consultas.



Versionar el esquema permite conocer cómo evolucionó la base.

El cambio afecta más que la base

Un cambio puede tener dependencias



También debemos revisar qué componentes utilizan la estructura modificada.

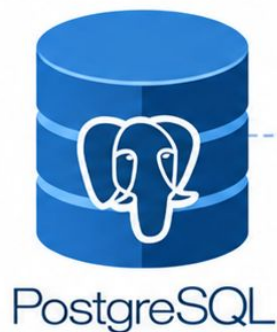


Un cambio pequeño en el esquema puede afectar varias partes del sistema.

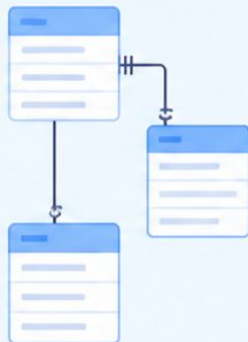
9

Práctica Guiada

Revisar, optimizar y evolucionar la base del sistema IoT.



Modelo y normalización



Índices

```
CREATE INDEX  
idx_mediciones_dispositivo  
ON mediciones  
(dispositivo_id,  
timestamp);
```



EXPLAIN ANALYZE

```
EXPLAIN ANALYZE  
SELECT * FROM mediciones  
WHERE dispositivo_id = 1  
ORDER BY timestamp DESC  
LIMIT 100;
```

```
Limit (cost=0.42..5.21 rows=100)  
-> Index Scan using idx_mediciones_..  
    (cost=0.42..1234.56 rows=1000)  
    (actual time=0.028..0.120 rows=100)  
Planning Time: 0.123 ms  
Execution Time: 0.152 ms
```

Aplicación y reportes



Práctica

Del esquema funcional a un esquema mejorado

Vamos a:

- detectar redundancia;
- normalizar una estructura;
- incorporar un nuevo requerimiento;
- decidir entre columna, relación y JSONB;
- analizar una consulta;
- crear un índice;
- comparar EXPLAIN ANALYZE antes y después;
- implementar una migración.



El objetivo es aplicar decisiones y comprobar sus efectos.

Paso 1

Detectar problemas de diseño

Partiremos de una estructura desnormalizada.

Vamos a identificar:

- información repetida;
- dependencias;
- problemas de actualización;
- problemas de inserción o eliminación.

dispositivo_id	dispositivo_nombre	ubicacion_nombre	organización_nombre	variable	valor	timestamp
D001	Sensor Temperatura	Laboratorio A	Facultad de Ciencias	temperatura	22.5	2024-04-01 10:00
D001	Sensor Temperatura	Laboratorio A	Facultad de Ciencias	humedad	45.1	2024-04-01 10:05
D001	Sensor Temperatura	Laboratorio A	Facultad de Ciencias	presion	1013	2024-04-01 10:10
D002	Estación Clima	Invernadero	Facultad de Agronomía	temperatura	18.2	2024-04-01 10:00
D002	Estación Clima	Invernadero	Facultad de Agronomía	humedad	50.0	2024-04-01 10:05
D003	Sensor Humedad	Laboratorio A	Facultad de Ciencias	humedad	46.7	2024-04-01 10:00
D003	Sensor Humedad	Laboratorio A	Facultad de Ciencias	bateria	95	2024-04-01 10:15

Diagram illustrating data redundancy and dependencies in a denormalized structure:

- Información repetida** (Redundant Information): Points to the first three rows (D001) where the same device (D001) and location (Laboratorio A) are repeated, indicating redundancy.
- Dependencia** (Dependency): Points to the last two rows (D003) where the same device (D003) and location (Laboratorio A) are repeated, indicating a dependency.



Antes de normalizar debemos comprender qué problema estamos resolviendo.

Paso 2

Normalizar y agregar un requerimiento

Vamos a transformar la estructura aplicando:

- 1FN;
- 2FN;
- 3FN.

Después incorporaremos:

Distintos dispositivos pueden poseer configuraciones diferentes.

Y decidiremos si corresponde utilizar:

- columnas;
- relaciones;
- JSONB.

Estructura inicial
(no normalizada)

id	nombre	tipo	configuracion
D001	Sensor A	Temperatura	{...}
D002	Sensor B	Humedad	{...}
D003	Estación C	Clima	{...}



Modelo normalizado
(tablas relacionadas)

dispositivo		tipo_dispositivo	
id	nombre	id	nombre
D001	Sensor A	T01	Temperatura
D002	Sensor B	T02	Humedad
D003	Estación C	T03	Clima

configuracion_dispositivo			
id	dispositivo_id	parametro	valor
C001	D001	intervalo	60
C002	D001	umbral	25
C003	D002	intervalo	10
C004	D003	modo	automatico



Alternativas para almacenar la configuración



Columna

Agregar columnas específicas en la tabla.



Relación

Crear tablas relacionadas para las configuraciones.



JSONB

Almacenar la configuración en un campo JSONB.



El diseño debe justificar dónde se almacena cada dato.

Paso 3

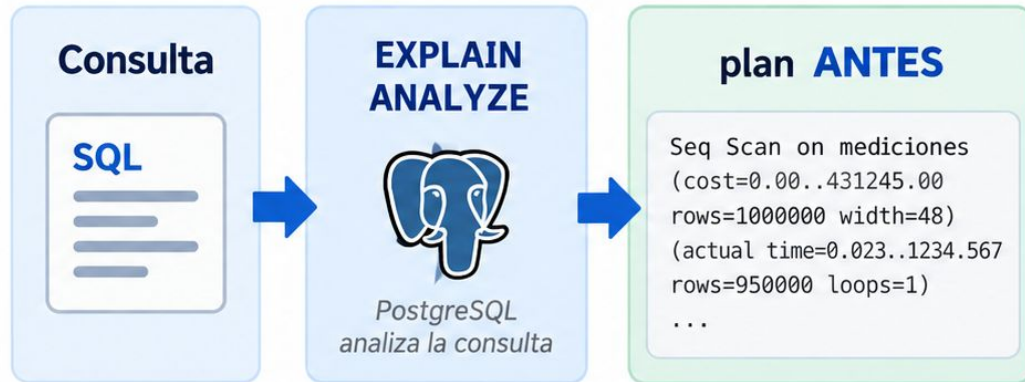
Medir una consulta frecuente

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = ?  
      AND timestamp >= ?  
      AND timestamp < ?  
ORDER BY timestamp;
```

Ejecutaremos:

```
EXPLAIN ANALYZE  
SELECT ...
```

y observaremos el comportamiento inicial.



Antes de optimizar necesitamos medir.

Paso 4

Crear el índice y comparar

```
CREATE INDEX idx_mediciones_dispositivo_timestamp  
ON mediciones (dispositivo_id, timestamp);
```

Volveremos a ejecutar:

```
EXPLAIN ANALYZE  
SELECT ...
```

y compararemos:

ANTES ↔ DESPUÉS

Crear índice



El índice acelera
la búsqueda en la
tabla.

EXPLAIN
ANALYZE

SQL

Analizamos el plan
de ejecución.

ANTES

Seq Scan on mediciones
(cost=0.00..431245.00)
(actual time=1234.567..
5678.901 rows=950000)
...

**Más tiempo
Más lecturas**

DESPUÉS

Index Scan using
idx_mediciones_...
on mediciones
(cost=0.42..1234.00)
(actual time=0.123..4.567
rows=1000)
...

**Menos tiempo
Menos lecturas**



La optimización debe comprobarse, no suponerse.

Paso 5

Evolucionar el esquema

Incorporaremos un nuevo dato como: `firmware_version`

Vamos a:

- modificar la estructura;
- considerar las filas existentes;
- verificar el nuevo esquema;
- identificar qué consultas pueden verse afectadas;
- registrar conceptualmente una nueva versión.

v1 — Esquema actual

Tabla dispositivos

dispositivo_id	nombre	numero_serie
1	Sensor Aula 1	SN-001
2	Estación Exterior	SN-002
3	Sensor Laboratorio	SN-003
4	Sensor Depósito	SN-004
5	Estación Patio	SN-005
...	...	...

Migración

Modificar la estructura



```
ALTER TABLE dispositivos  
ADD COLUMN  
firmware_version TEXT;
```

- ✓ Aplicar migración
- ✓ Considerar datos existentes
- ✓ Verificar esquema
- ✓ Actualizar consultas
- ✓ Registrar nueva versión

v2 — Esquema actualizado

Tabla dispositivos

dispositivo_id	nombre	numero_serie	firmware_version
1	Sensor Aula 1	SN-001	1.0.3
2	Estación Exterior	SN-002	2.1.0
3	Sensor Laboratorio	SN-003	1.0.1
4	Sensor Depósito	SN-004	1.2.0
5	Estación Patio	SN-005	2.0.5
...	...	...	...



Una migración conecta la evolución del requerimiento con la evolución de la base.



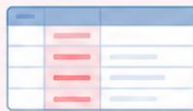
Para ir cerrando..

Errores frecuentes

Problemas típicos de diseño y optimización

- ❗ Repetir información sin necesidad.
- ❗ Normalizar mecánicamente.
- ❗ Desnormalizar sin un problema concreto.
- ❗ Crear índices para todas las columnas.
- ❗ Suponer que un índice siempre se utilizará.
- ❗ Optimizar sin medir.
- ❗ Considerar todo Seq Scan como un problema.
- ❗ Guardar todo dentro de JSONB.
- ❗ Cambiar el esquema sin considerar los datos existentes.

Información repetida



Normalización mecánica



Desnormalización sin un problema concreto



Índices innecesarios



```
CREATE INDEX ...  
CREATE INDEX ...  
CREATE INDEX ...
```

El índice no siempre se utiliza

```
Seq Scan on mediciones  
(cost=0.00..4312.00  
rows=100000)
```

Optimizar sin medir



Todo Seq Scan es un problema

```
Seq Scan  
(cost=0.00..1234.00  
rows=10000)
```

Todo en JSONB



Cambios sin considerar datos existentes

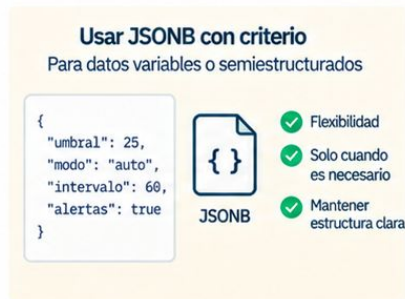
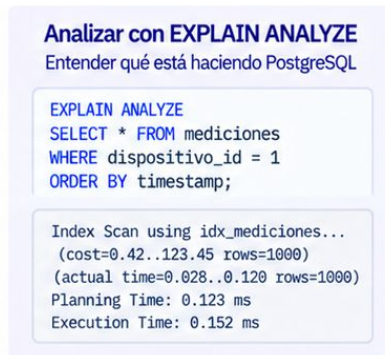


Optimizar no significa aplicar técnicas automáticamente. Significa tomar decisiones a partir del problema.

Buenas prácticas

Diseñar y optimizar con criterio

- ✓ Analizar dependencias.
- ✓ Reducir redundancia innecesaria.
- ✓ Desnormalizar solamente con una razón.
- ✓ Diseñar índices desde las consultas.
- ✓ Considerar selectividad y costo.
- ✓ Utilizar EXPLAIN ANALYZE.
- ✓ Utilizar JSONB ante variabilidad real.
- ✓ Versionar los cambios del esquema.
- ✓ Medir antes y después de optimizar.



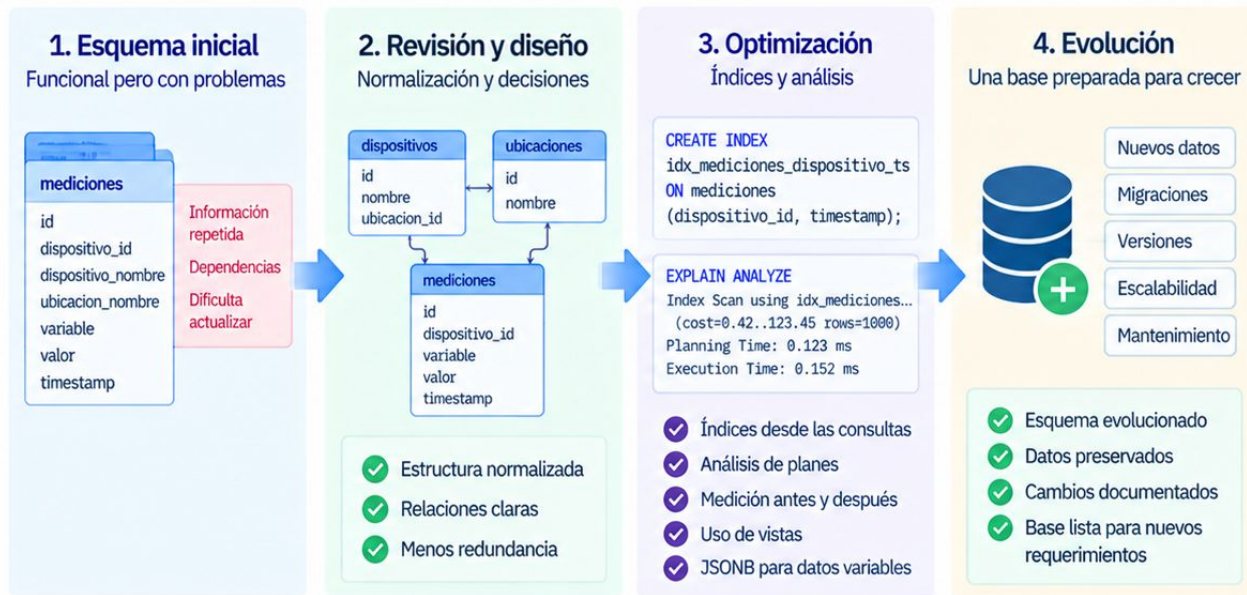
Una buena base debe poder mantenerse, medirse y evolucionar.

¿Qué observamos?

De una base funcional a una base que podemos evaluar

Durante la clase:

- detectamos redundancia;
- normalizamos estructuras;
- analizamos desnormalización;
- diseñamos índices;
- estudiamos planes de ejecución;
- medimos antes y después;
- utilizamos vistas;
- incorporamos JSONB;
- evolucionamos el esquema.

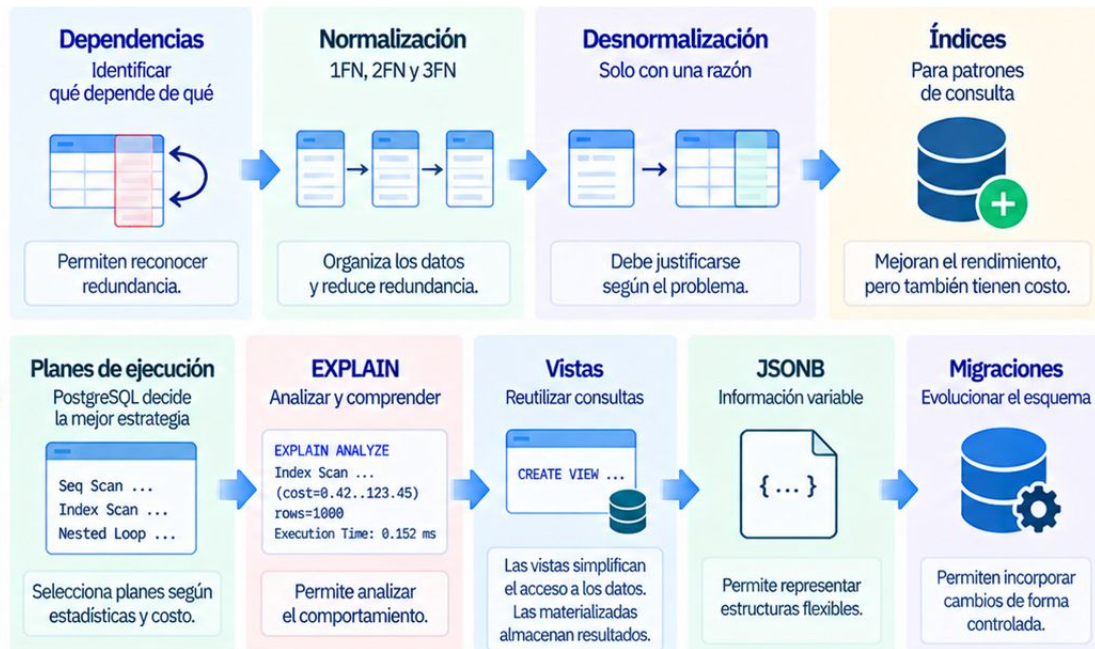


Diseñar una base no termina cuando las tablas comienzan a funcionar.

Resumen de la clase

Ideas principales

- Las dependencias permiten reconocer redundancia.
- 1FN, 2FN y 3FN ayudan a organizar los datos.
- La desnormalización debe justificarse.
- Los índices responden a patrones de consulta.
- La selectividad influye en su utilidad.
- Los índices también tienen costo.
- PostgreSQL selecciona planes de ejecución.
- EXPLAIN y EXPLAIN ANALYZE permiten analizarlos.
- Las vistas reutilizan consultas.
- Las vistas materializadas almacenan resultados.
- JSONB permite representar información variable.
- Las migraciones permiten evolucionar el esquema.



Una base bien diseñada debe mantener consistencia, responder eficientemente y poder evolucionar.

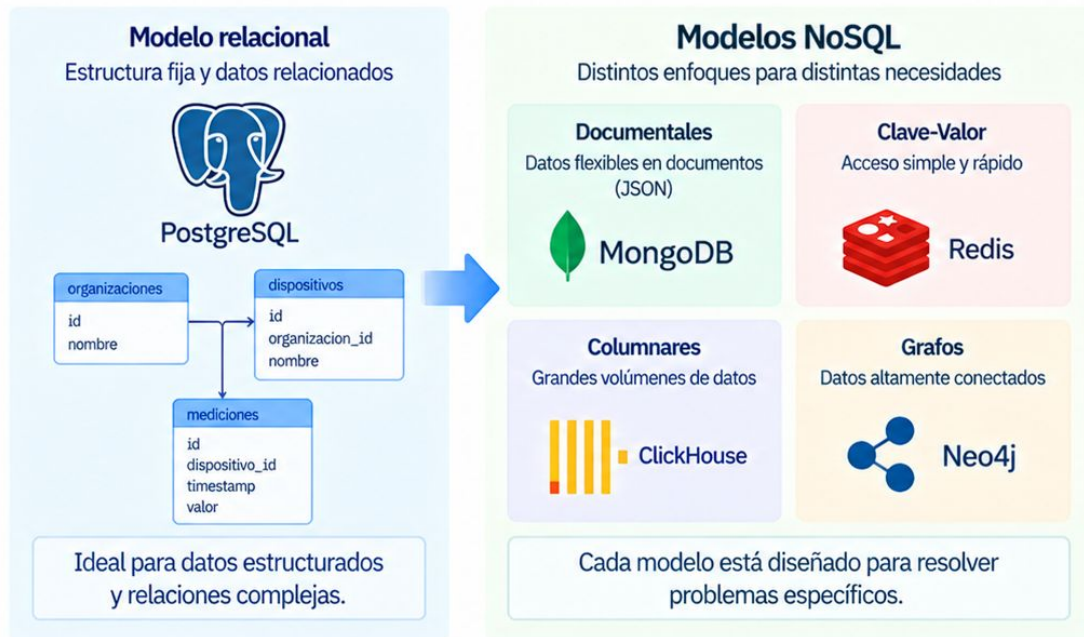
Próxima clase

Bases de Datos NoSQL

Hasta ahora trabajamos principalmente con el modelo relacional.

En la próxima clase vamos a analizar:

- por qué aparecieron las bases NoSQL;
- flexibilidad de esquema;
- bases documentales;
- clave-valor;
- bases columnares;
- bases de grafos;
- consistencia y disponibilidad;
- criterios para elegir entre SQL y NoSQL.



Hoy aprendimos a mejorar y evolucionar una base relacional.
La próxima clase ampliaremos el conjunto de modelos disponibles.

Evaluación 2

Clases 3 y 4

Clase 3

Bases de datos relacionales y SQL

- modelo relacional;
- SQL;
- JOIN;
- agregaciones;
- transacciones.



```
SELECT ...  
FROM ...  
JOIN ...  
GROUP BY ...  
BEGIN;  
COMMIT;
```

Clase 4

Diseño relacional, optimización y evolución

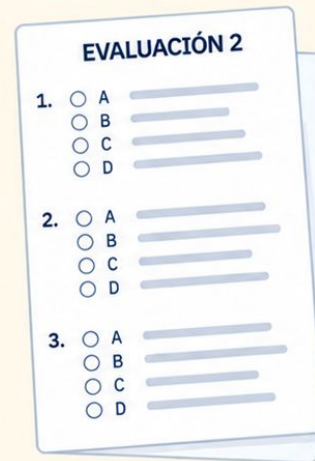
- normalización;
- índices;
- planes de ejecución;
- vistas;
- JSONB;
- evolución del esquema.



```
CREATE INDEX ...  
EXPLAIN ANALYZE ...  
CREATE VIEW ...  
{ ... } -- JSONB  
ALTER TABLE ...
```

Evaluación 2

Clases 3 y 4



Comprensión

Entender conceptos y sus relaciones.



Interpretación

Analizar situaciones y resultados.



Aplicación

Resolver problemas en contextos reales.

Las preguntas combinarán comprensión, interpretación y aplicación.



El objetivo es comprender decisiones de diseño y consulta, no solamente recordar *sintaxis*.



Enlace a la encuesta



Muchas gracias ¿Dudas?